分散式訓練

Distributed Training Deep Dive

從集合通訊、資料平行到模型平行 — 大型語言模型的核心訓練技術

課程大綱

Course Outline

Part 1	導論與動機	Scaling Law、三大核心問題
Part 2	集合通訊入門	Rank/World Size + 五種原語 + 練習
Part 3	資料平行 DP DDP FSDP	nn.DataParallel、梯度累積、同步 vs 非同步
Part 4	模型平行 TP	張量平行、排程策略
Part 5	模型平行 PP	管線平行、排程策略
Part 6	混合式訓練	3D/4D Parallelism、實戰配置與案例

Part 1

導論與動機

為什麼需要分散式訓練？

?

為什麼現在的 AI 模型越來越大？

Llama-2 7B → Llama-3.1 405B → DeepSeek-V3 671B (MoE) → GPT-4 ~1.8T (MoE)
更多參數帶來了什麼好處？代價是什麼？

Scaling Law：模型越大效果越好，但訓練代價也急速上升

為什麼模型越來越大？

模型表現隨規模提升

參數量、資料量、算力三者共同驅動效能 (Scaling Law)

更大模型出現湧現能力

大模型展現強泛化、推理與 in-context learning

近年規模快速擴張

Llama-3.1 405B → DeepSeek-V3 671B → GPT-4 ~1.8T (MoE)

模型變大的代價

VRAM

單張 GPU 放不下完整模型

Llama-3.1 405B FP16 需 ~810GB，H100 僅 80GB

Time

需大量 GPU、長時間訓練

Llama-3.1 405B 耗費 ~30.8M H100 GPU-hours

Cost

訓練成本極高

GPT-4 級 ~$1 億；DeepSeek-V3 MoE 僅 ~$560 萬

因此：模型規模持續增加，單機單卡已難以支撐 → 分散式訓練成為必要選擇

分散式訓練的三大核心問題

Memory / Compute / Communication

記憶體

Memory

模型參數太大，VRAM 不夠

參數+梯度+Optimizer+Activation

70B FP16+Adam → ~1120GB (14 張 H100)

解法：ZeRO、FSDP、Offload

計算

Compute

海量矩陣運算

單 GPU 算太慢

需要多種平行策略分擔

解法：DP / TP / PP / SP

通訊

Communication

GPU 間頻繁交換資料

梯度同步 (All-Reduce)

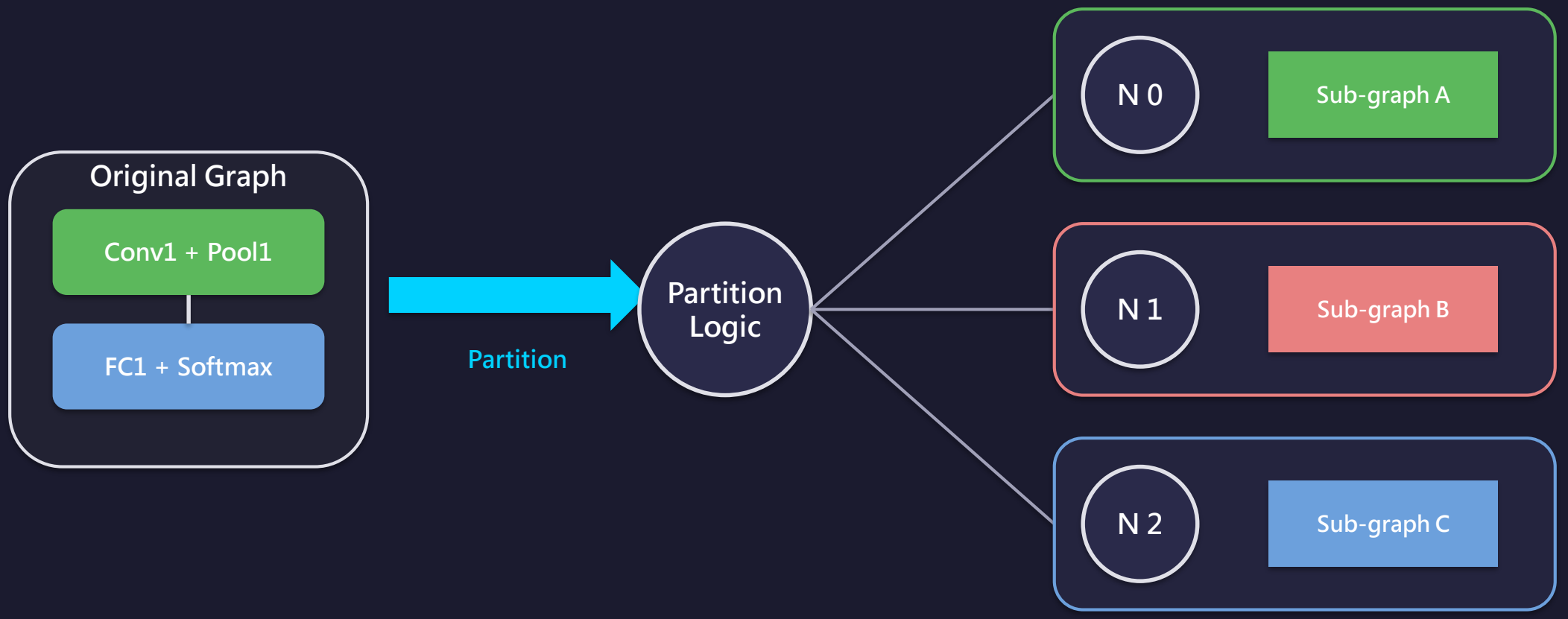
Activation 傳遞

解法：通算重疊、Ring All-Reduce

訓練耗時 = (資料量 x 計算量) / (單裝置速率 x 裝置數 x 平行效率)

分散式訓練系統

計算圖跨節點切分 — Partition Graph → Dispatch Partitions



分散式訓練系統

計算圖跨節點同步數據（權重 weights）信息

All Nodes Sync Local Data / Gradients

示例：分散式 MNIST — Synchronous SGD

用 torch.distributed 實作分散式隨機梯度下降

- 示例：分散式 MNIST

每個 Worker 各自做 forward + backward
算完 local gradient 後，透過
all_reduce 同步梯度 → 取平均
→ 確保所有 Worker 的模型一致

核心函式：average_gradients

```
""" Gradient averaging. """
def average_gradients(model):
    size = float(dist.get_world_size())
    for param in model.parameters():
        dist.all_reduce(
            param.grad.data,
            op=dist.reduce_op.SUM)
        param.grad.data /= size
```

```
""" Distributed Synchronous SGD Example """
def run(rank, size):
    torch.manual_seed(1234)
    train_set, bsz = partition_dataset()
    model = Net()
    optimizer = optim.SGD(
        model.parameters(),
        lr=0.01, momentum=0.5)

    num_batches = ceil(
        len(train_set.dataset) / float(bsz))
    for epoch in range(10):
        epoch_loss = 0.0
        for data, target in train_set:
            optimizer.zero_grad()
            output = model(data)
            loss = F.nll_loss(output, target)
            epoch_loss += loss.item()
            loss.backward()
            average_gradients(model)  # ← 關鍵
            optimizer.step()
```

Part 2

集合通訊

八種通訊原語 — 分散式訓練的基礎工具

集合式通信方式

Collective Communication — 分散式訓練的通訊基礎

一對多

Scatter / Broadcast

一個節點把資料發送給所有其他節點
- Broadcast：同一份資料 → 全部
- Scatter：不同份資料 → 分別給不同節點

多對一

Gather / Reduce

所有節點把資料彙集到一個節點
- Gather：收集原始資料
- Reduce：收集並歸約（如 SUM）

多對多

All-Reduce / All-Gather / Reduce-Scatter / All-to-All

所有節點互相交換資料
- All-Reduce：全部歸約，每人都拿到結果
- All-Gather：全部收集，每人都有完整資料
- Reduce-Scatter：歸約後分片
- All-to-All：全對全轉置交換

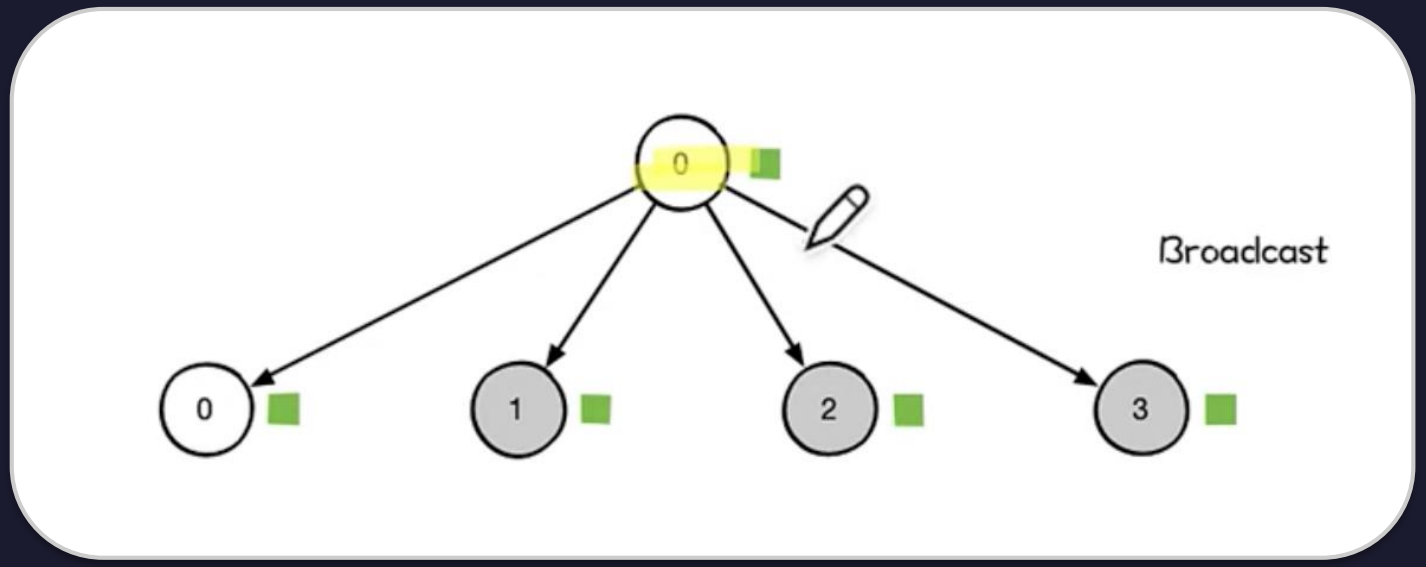
集中式通信方式 (I)：一對多 Broadcast

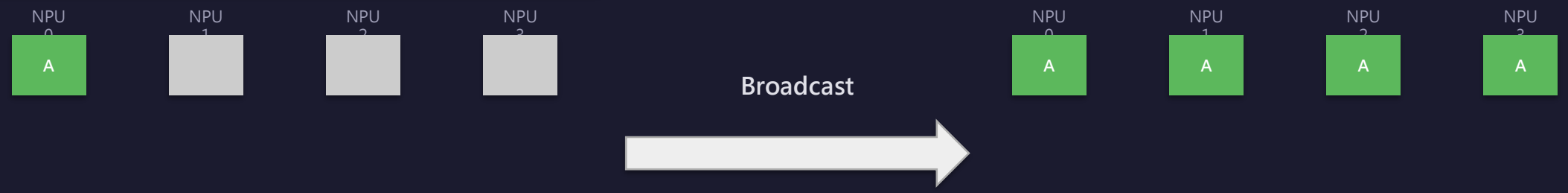
某個節點把自身的數據發送到集群中的其他節點

分散式機器學習中常用於網絡參數的初始化。

一個來源 rank 的資料，複製到所有節點。

用途：同步初始參數、模型權重初始化





Broadcast — PyTorch 程式範例

torch.distributed API 實戰

核心 API 呼叫

```
dist.broadcast(tensor, src=0)
```

說明

src=0 表示資料來源是 rank 0
所有 rank 呼叫同一行 broadcast
呼叫結束後，每個 rank 的 tensor
都會變成 src 的值

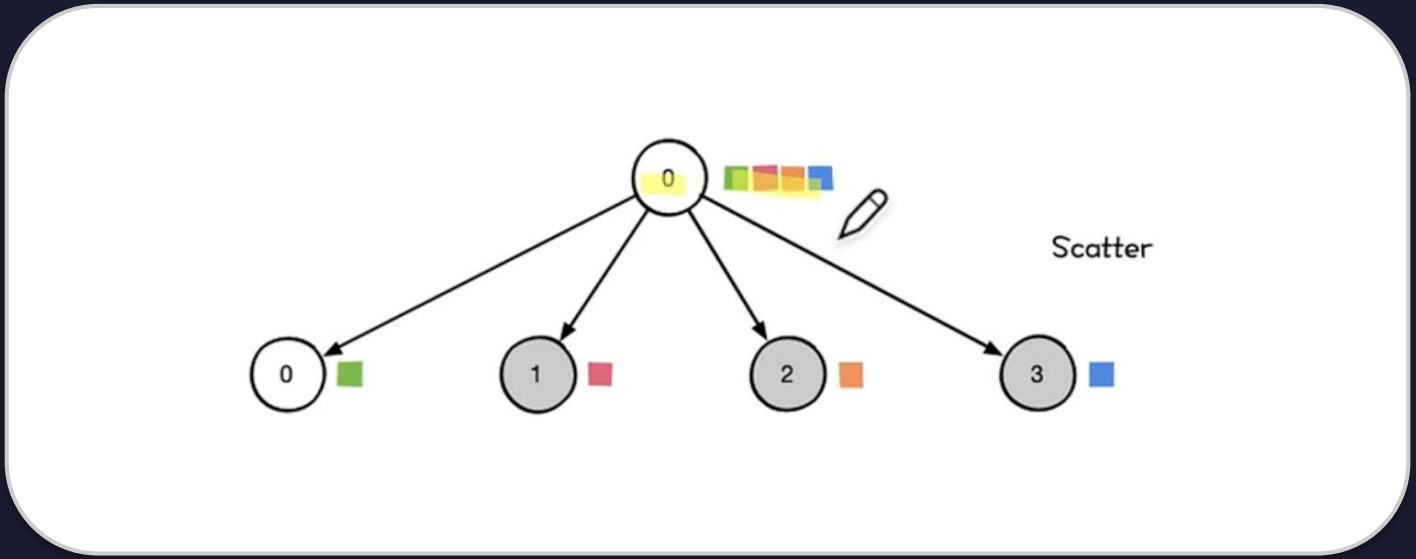
```
import torch
import torch.distributed as dist

dist.init_process_group('nccl')
rank = dist.get_rank()

# Rank 0 has the model weights
if rank == 0:
    tensor = torch.tensor([1.0, 2.0, 3.0],
                          device='cuda')
else:
    tensor = torch.zeros(3, device='cuda')

# Broadcast from rank 0 to all ranks
dist.broadcast(tensor, src=0)

# Now ALL ranks have [1.0, 2.0, 3.0]
```

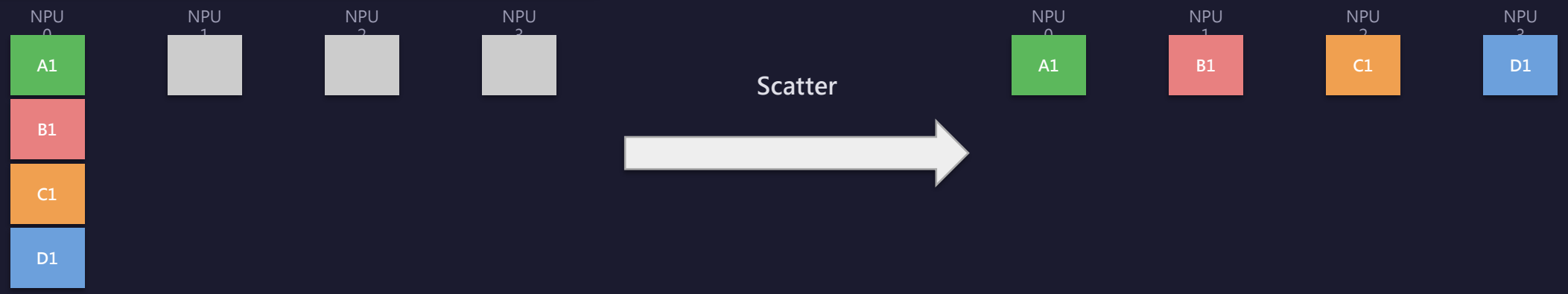
集中式通信方式 (I)：一對多 Scatter

將主節點的數據進行劃分並散布至其他指定的節點

主節點有完整資料，切開後分配給不同 rank。每個 rank 拿到不同的資料片段。

用途：分配訓練資料、分割 tensor





Scatter — PyTorch 程式範例

torch.distributed API 實戰

核心 API 呼叫

```
dist.scatter(output, scatter_list, src=0)
```

說明

scatter_list 只有 src rank 需要提供
其他 rank 傳 None 即可
每個 rank 的 output 會被填入
scatter_list 中對應的那一份

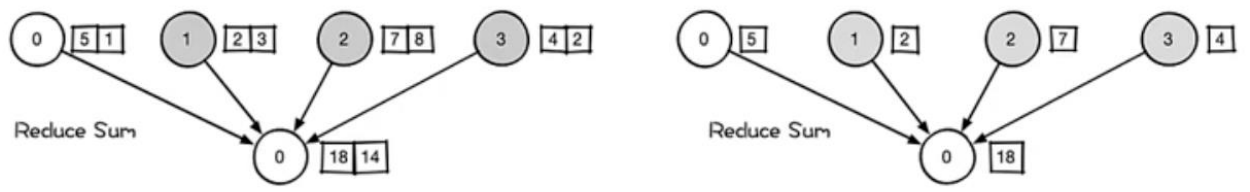
```
import torch
import torch.distributed as dist

rank = dist.get_rank()
world_size = dist.get_world_size()

output = torch.zeros(1, device='cuda')

if rank == 0:
    scatter_list = [
        torch.tensor([float(i+1)],
                     device='cuda')
        for i in range(world_size)]
else:
    scatter_list = None

dist.scatter(output, scatter_list, src=0)
# rank0:[1] rank1:[2] rank2:[3] rank3:[4]
```

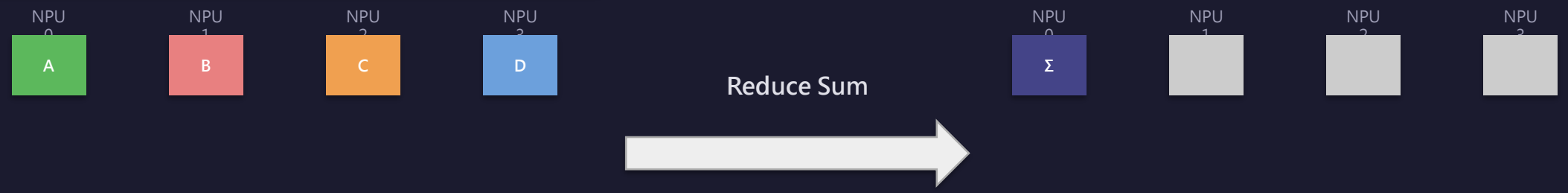
集中式通信方式 (II)：多對一 Reduce

Reduce 稱為規約運算，是一系列簡單運算操作的統稱

所有 rank 的資料做歸約運算
（SUM / MIN / MAX / PROD 等），
結果只存在 dst rank。

用途：集中計算全局 loss、統計量





Reduce — PyTorch 程式範例

torch.distributed API 實戰

核心 API 呼叫

```
dist.reduce(tensor, dst=0,
op=dist.ReduceOp.SUM)
```

說明

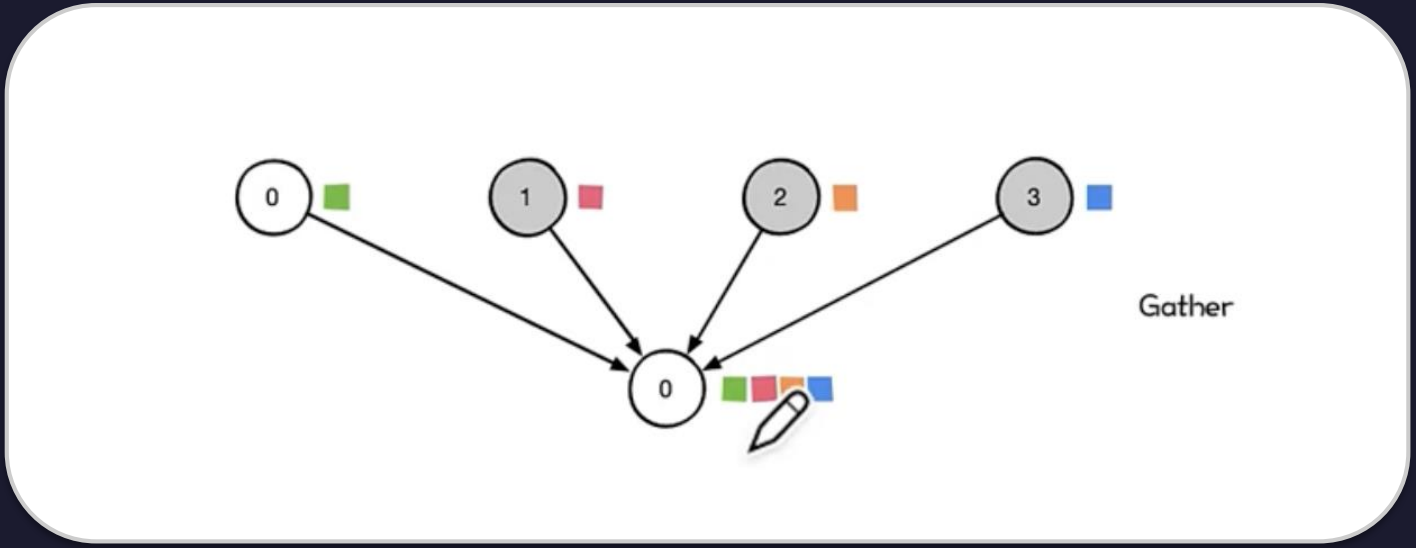
dst=0 表示結果只存到 rank 0
所有 rank 都要呼叫 reduce
op 可選：SUM / PRODUCT / MIN / MAX
注意：只有 dst rank 的 tensor 有結果

```
import torch
import torch.distributed as dist

rank = dist.get_rank()
tensor = torch.tensor([float(rank + 1)],
                      device='cuda')
# rank0:[1] rank1:[2] rank2:[3] rank3:[4]

dist.reduce(tensor, dst=0,
            op=dist.ReduceOp.SUM)

# 只有 rank 0 的 tensor 變成 [10.0]
# 其他 rank 的 tensor 不變
```

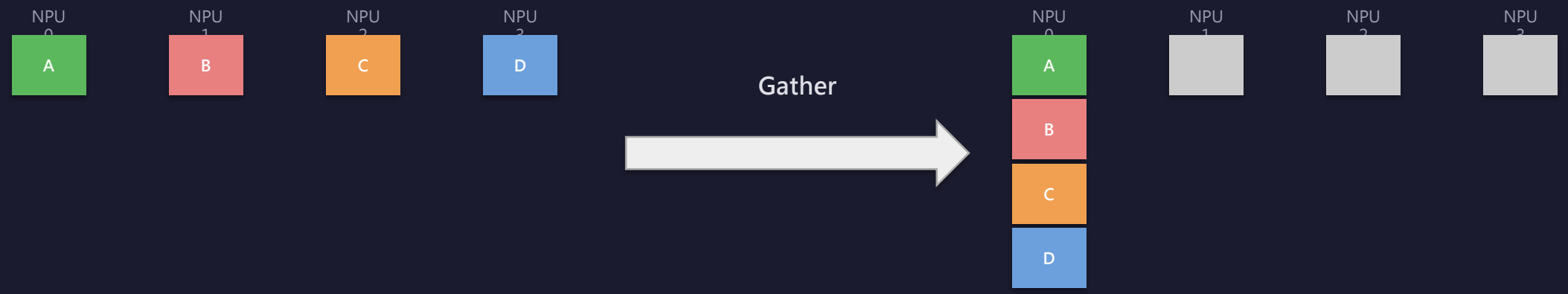
集中式通信方式 (II)：多對一 Gather

將多個 Sender 上的數據收集到單個節點上

Gather 可以理解為反向的 Scatter。
多個 rank 各自有一份資料，
全部收集到 dst rank。



用途：收集各 Worker 的預測結果、排序



Gather — PyTorch 程式範例

torch.distributed API 實戰

核心 API 呼叫

```
dist.gather(tensor, gather_list, dst=0)
```

說明

gather_list 只有 dst rank 需要提供
其他 rank 傳 None
dst rank 收到所有人的 tensor
按 rank 順序排列在 gather_list 中

```
import torch
import torch.distributed as dist

rank = dist.get_rank()
tensor = torch.tensor([float(rank + 1)],
                      device='cuda')

if rank == 0:
    gather_list = [
        torch.zeros(1, device='cuda')
        for _ in range(4)]
else:
    gather_list = None

dist.gather(tensor, gather_list, dst=0)
# rank0: gather_list=[[1],[2],[3],[4]]
```

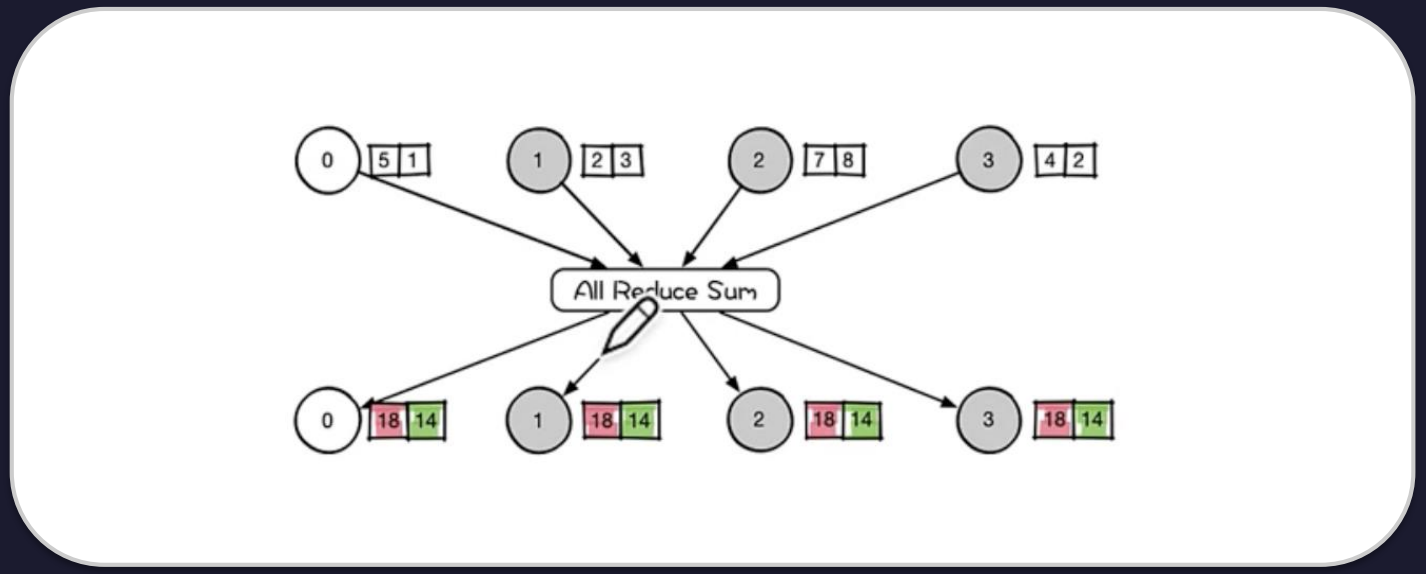
集中式通信方式 (III)：多對多 All-Reduce

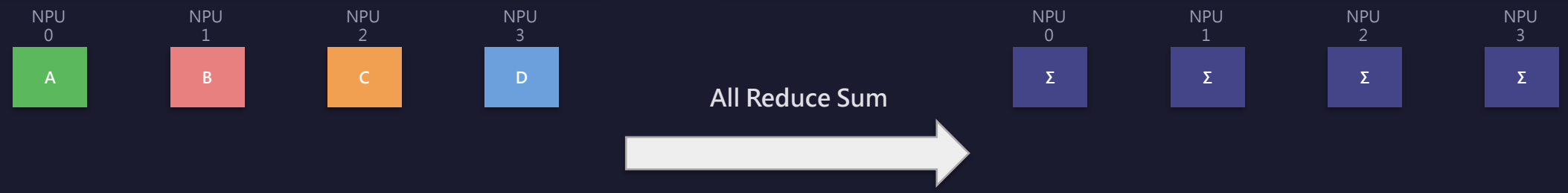
All Reduce 是在所有節點上都應用同樣的 Reduce 操作

= Reduce + Broadcast
= Reduce-Scatter + All-Gather

所有 rank 做歸約，而且每個 rank 都拿到完整的歸約結果。

用途：DP/DDP 梯度同步（最重要的操作！）





All-Reduce — PyTorch 程式範例

torch.distributed API 實戰

核心 API 呼叫

```
dist.all_reduce(grad,
op=dist.ReduceOp.SUM)
grad /= world_size
```

說明

這是分散式訓練最重要的操作！
DDP 就是在 backward 後呼叫
all_reduce 同步所有 Worker 的梯度
再除以 world_size 取平均

```
import torch
import torch.distributed as dist

rank = dist.get_rank()
world_size = dist.get_world_size()

grad = torch.tensor([float(rank + 1)],
                    device='cuda')

dist.all_reduce(grad,
                op=dist.ReduceOp.SUM)

grad /= world_size
# ALL ranks now have: [2.5]
```

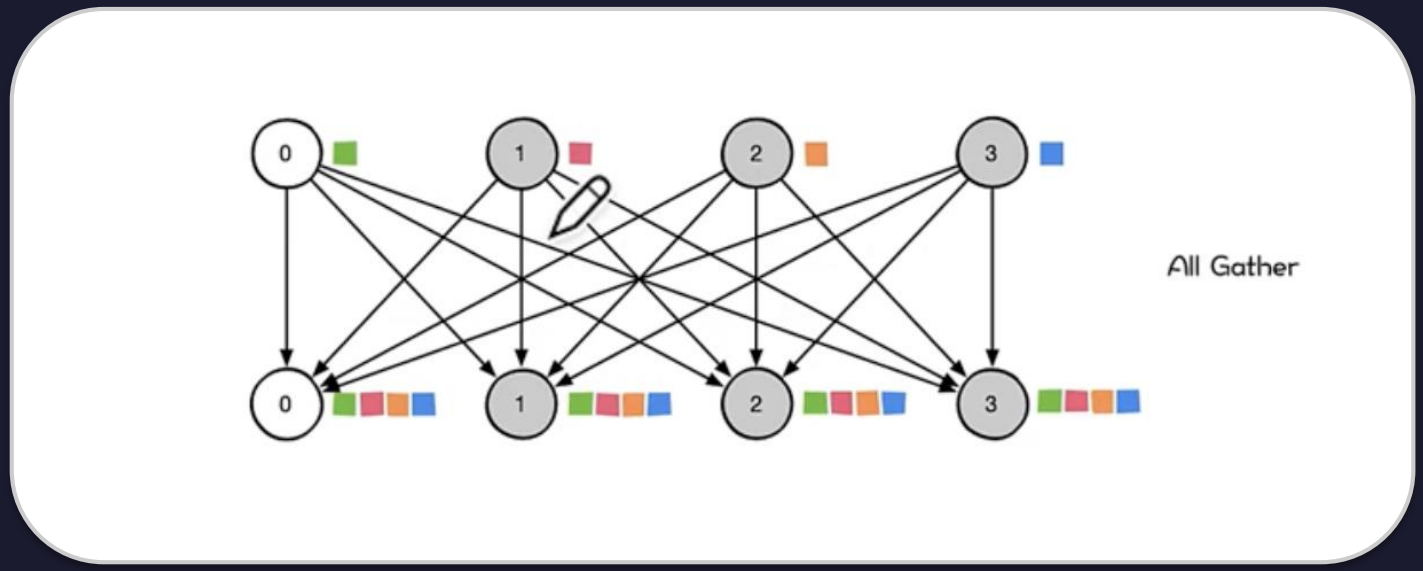
集中式通信方式 (III)：多對多 All-Gather

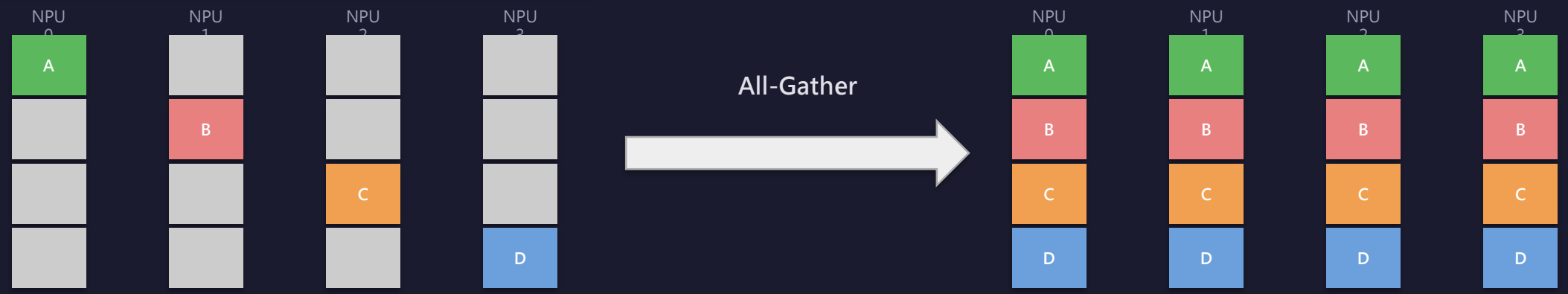
All Gather 會收集所有數據到所有節點上

All Gather = Gather + Broadcast

每個 rank 各有一份資料，
呼叫後所有 rank 都拿到完整的
所有人的資料。

用途：ZeRO-3 / FSDP 收集完整參數





All-Gather — PyTorch 程式範例

torch.distributed API 實戰

核心 API 呼叫

```
dist.all_gather(gathered, local)
```

說明

gathered 是一個 list of tensors
長度 = world_size（一人一份）
呼叫後每個 rank 的 gathered
都包含所有人的 local tensor

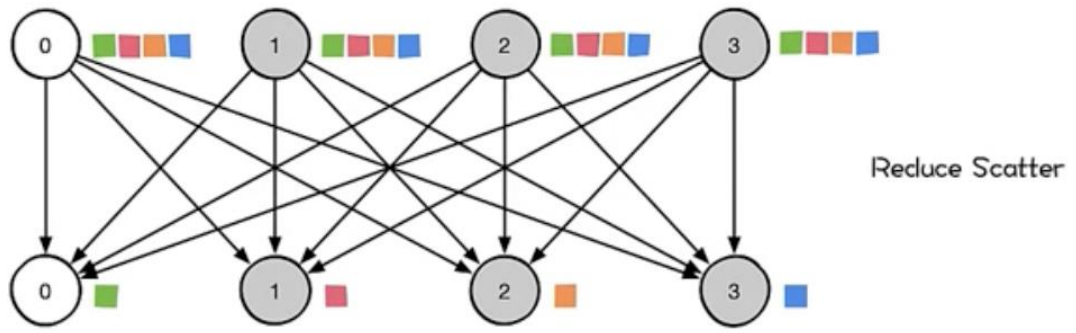
```
import torch
import torch.distributed as dist

rank = dist.get_rank()
world_size = dist.get_world_size()

local = torch.tensor([float(rank + 1)],
                     device='cuda')

gathered = [torch.zeros(1, device='cuda')
            for _ in range(world_size)]

dist.all_gather(gathered, local)
# ALL ranks: gathered=[[1],[2],[3],[4]]
```

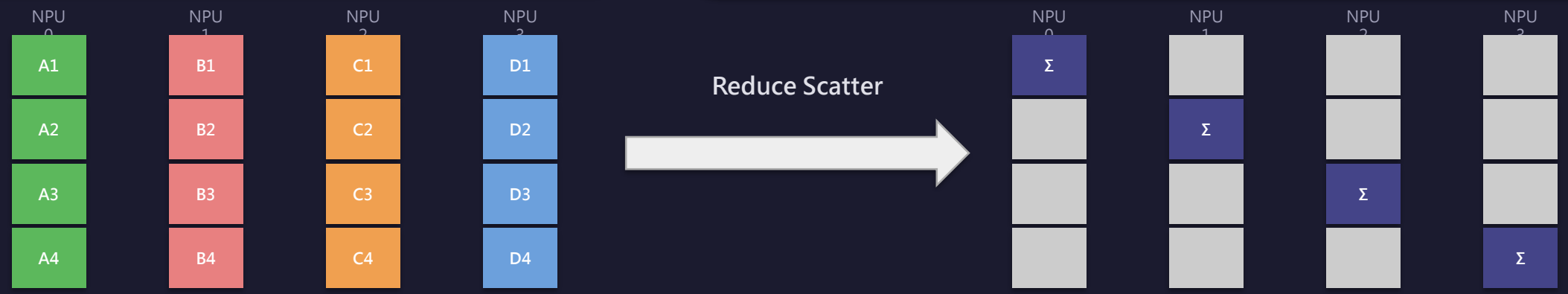
集中式通信方式 (III)：多對多 Reduce-Scatter

Reduce Scatter 會將各節點的輸入先進行求和，然後按卡數切分並分發

先歸約（如 SUM），再切開
分給不同 rank。每個 rank
只拿到結果的一個切片。

用途：ZeRO-2 / FSDP 梯度分區





Reduce-Scatter — PyTorch 程式範例

torch.distributed API 實戰

核心 API 呼叫

```
dist.reduce_scatter(output, input_list,
op=...)
```

說明

input_list: 長度=world_size 的 list
output: 一個 tensor（歸約後的分片）
比 All-Reduce 省記憶體：
每人只需存 1/N 的結果

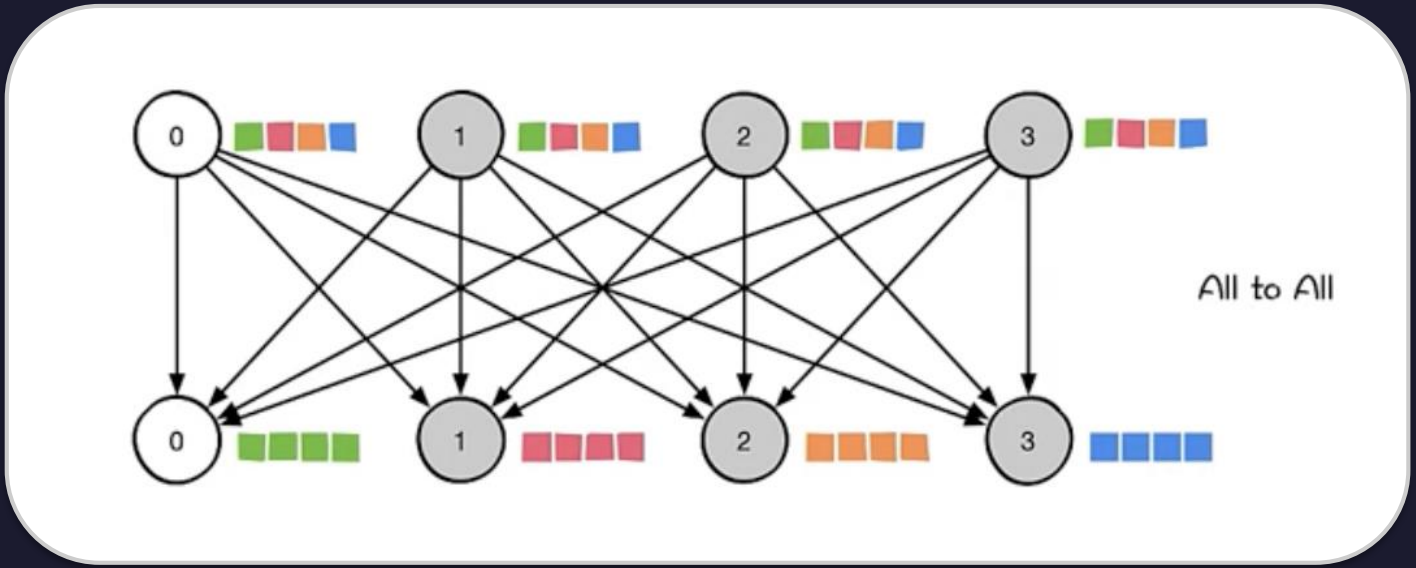
```
import torch
import torch.distributed as dist

rank = dist.get_rank()
world_size = dist.get_world_size()

input_tensor = torch.ones(world_size,
    device='cuda') * (rank + 1)

input_list = list(
    input_tensor.chunk(world_size))

output = torch.zeros(1, device='cuda')
dist.reduce_scatter(output, input_list,
    op=dist.ReduceOp.SUM)
# 每個 rank 只拿到一個 chunk 的 sum
```

集中式通信方式 (III)：多對多 All-to-All

將節點 i 的發送緩衝區中的第 j 塊數據發送給節點 j

相當於矩陣轉置：
rank i 的第 j 塊 → rank j。
每個 rank 向其他 rank 發送
不同的資料。

用途：MoE 專家路由、Ulysses SP



All-to-All — PyTorch 程式範例

torch.distributed API 實戰

核心 API 呼叫

```
dist.all_to_all(output_list, input_list)
```

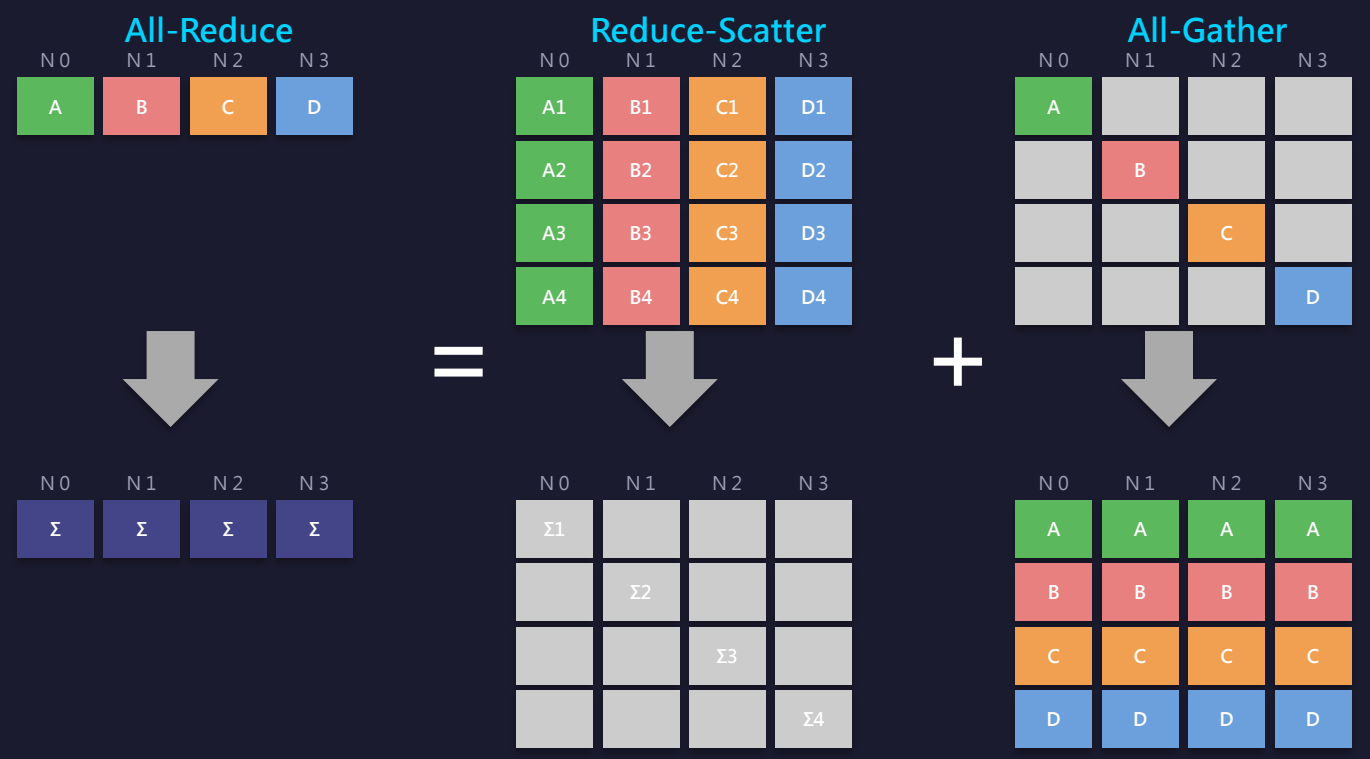
說明

input_list[j] = 要發給 rank j 的 tensor
output_list[j] = 從 rank j 收到的 tensor
像矩陣轉置：行→列
MoE 中用來把 token 送到正確的 Expert

```
import torch
import torch.distributed as dist

rank = dist.get_rank()
world_size = dist.get_world_size()

input_list = [
    torch.tensor([float((rank+1)*10 + i)],
                 device='cuda')
    for i in range(world_size)]

output_list = [
    torch.zeros(1, device='cuda')
    for _ in range(world_size)]

dist.all_to_all(output_list, input_list)
# rank i 收到每個 rank 發給 i 的那份
```

集中式通信方式 (III)：操作分解

All-Reduce = Reduce-Scatter + All-Gather



重要分解：
DDP 用完整的 All-Reduce (RS+AG)，每張卡都會拿到完整的梯度。

FSDP / ZeRO-2 為了節省記憶體，只做前段的 Reduce-Scatter，讓每張卡只保存自己負責的那一塊梯度切片。

集合通訊總結

Summary

1. **了解集合式通信的 3 種不同方式**

   一對多（Scatter / Broadcast）、多對一（Gather / Reduce）、多對多（All-Reduce 等）

2. **了解一對多 Scatter/Broadcast，多對一 Gather/Reduce，多對多的具體方式**

   All-Reduce / All-Gather / Reduce-Scatter / All-to-All 的資料流動與 API 用法

3. **了解多對多可以由一對多和多對一的方式組合**

   All-Reduce = Reduce-Scatter + All-Gather（DDP 使用完整 All-Reduce，FSDP 只做 Reduce-Scatter 省記憶體）

練習作業 1.1

不用 torch.distributed，先理解資料流再學 API

題目 1.1：用純 Python 實作 all_gather_py(rank_data)

程式骨架

```
def all_gather_py(rank_data):
    """
    每個 rank 都拿到所有人的資料。
    rank_data: list of lists，一個 rank 一份
    """
    # TODO: 實作
    pass

# 測試
rank_data = [[1], [2], [3], [4]]
result = all_gather_py(rank_data)
# 每個 rank 都應得到 [1,2,3,4]
```

輸入

```
rank_data = [[1], [2], [3], [4]]
```

預期輸出（每個 rank 都得到）

```
rank0: [1, 2, 3, 4]
rank1: [1, 2, 3, 4]
rank2: [1, 2, 3, 4]
rank3: [1, 2, 3, 4]
```

概念提示

All-Gather 的本質：把所有人各自的那份資料「串接」成一份完整清單，每個人都保存一份副本。

練習作業 1.2

不用 torch.distributed，先理解資料流再學 API

題目 1.2：用純 Python 實作 reduce_scatter_sum_py(rank_data)

程式骨架

```
def reduce_scatter_sum_py(rank_data):
    """
    先逐元素 SUM，再切開每人只拿一份。
    rank_data: list of lists，每份長度 = world_size
    """
    # TODO: 實作
    pass

# 測試
rank_data = [[1,1,1,1], [2,2,2,2],
             [3,3,3,3], [4,4,4,4]]
result = reduce_scatter_sum_py(rank_data)
```

輸入

```
rank_data = [[1,1,1,1], [2,2,2,2],
             [3,3,3,3], [4,4,4,4]]
```

預期輸出（每人只拿一份）

```
rank0: [10]  rank1: [10]
rank2: [10]  rank3: [10]
```

概念提示

Reduce-Scatter 兩步走：① 逐元素加總 [1+2+3+4, ...] = [10,10,10,10] ② 切開每人只拿 index == rank 那格。

練習作業 1.3

用 1.1 + 1.2 拼出 All-Reduce

題目 1.3：呼叫你自己寫的 reduce_scatter → all_gather 實作 all_reduce

程式骨架

```
def all_reduce_via_two_steps(rank_data):
    """
    呼叫你寫的：
      - reduce_scatter_sum_py
      - all_gather_py
    """
    # Step 1: 先 Reduce-Scatter
    # TODO

    # Step 2: 再 All-Gather
    # TODO
    pass
```

輸入

```
rank_data = [[1,1,1,1], [2,2,2,2],
             [3,3,3,3], [4,4,4,4]]
```

預期輸出（每人都拿到全部）

```
rank0: [10,10,10,10]
rank1: [10,10,10,10]
rank2: [10,10,10,10]
rank3: [10,10,10,10]
```

Reduce-Scatter + All-Gather = All-Reduce

Part 3

Data Parallelism

資料平行 — DP → DDP → FSDP 的演化

Data Parallelism

資料平行概覽

1. Data parallelism, DP
2. Distributed Data Parallel, DDP
3. Fully Sharded Data Parallel, FSDP

Data Parallelism

為什麼需要資料平行？

1. Data parallelism, DP
2. Distributed Data Parallel, DDP
3. Fully Sharded Data Parallel, FSDP

影像：256 x 256 x 3

NLP：數十億 Token (15T+ tokens for Llama-3)

大規模資料 → 單張 GPU 訓練太慢

前沿模型使用 15T+ tokens 預訓練 (Llama-3.1)
Llama-3.1 405B 需 ~30.8M H100 GPU-hours

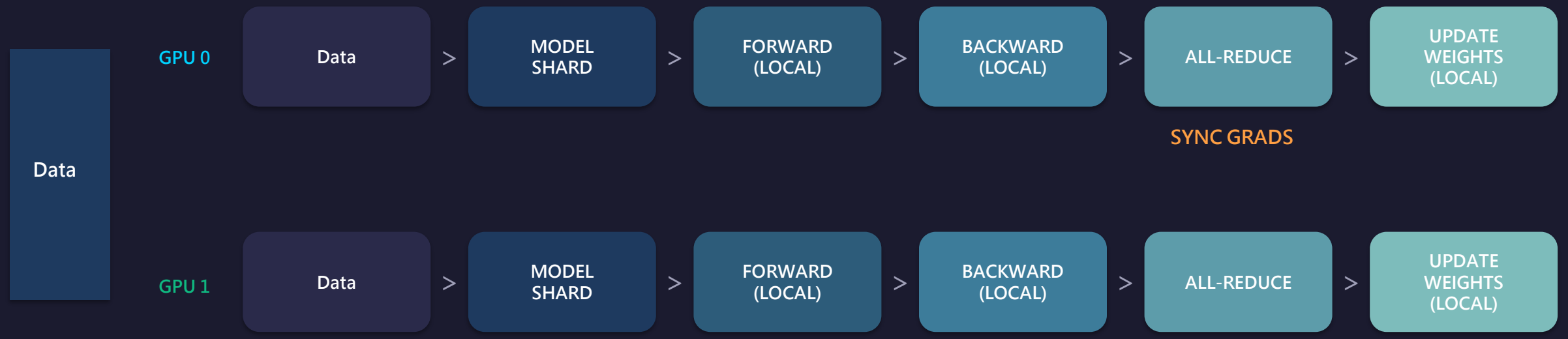
解法：把資料切成 N 份，每張 GPU 各拿一份同時訓練
→ 理想加速比 = N（實際會因為通訊開銷略低）

關鍵前提：模型放得進一張 GPU 的 VRAM

Data Parallelism, DP

Data Parallel 自動切分資料並同步梯度

Data Parallel automatically splits training data and sends model jobs to multiple GPUs.
After each model completed, Data Parallel will Accumulate Gradients.



Data Parallelism Limitation (I)

Practice — nn.DataParallel 實作

Practice

Simple implementation of code logic.

Multithreading parallel. Controlled by a process, restricted by GIL.

```
device = torch.device("cuda:1,3")  ## specify the GPU id's

model = CreateModel()

model = nn.DataParallel(model, device_ids=[1, 3])
model.to(device)
```

GIL (Global Interpreter Lock)：Python 同一時間只允許一個執行緒執行 bytecode
→ 多執行緒架構無法真正利用多顆 CPU/GPU，等同假平行

Data Parallelism Limitation (II)

Theoretical — 梯度累積的兩大問題

Theoretical

Models of each machine are independent, clusters using computing power.

Each machine maintain self gradient, accumulate gradients using Collective communication.

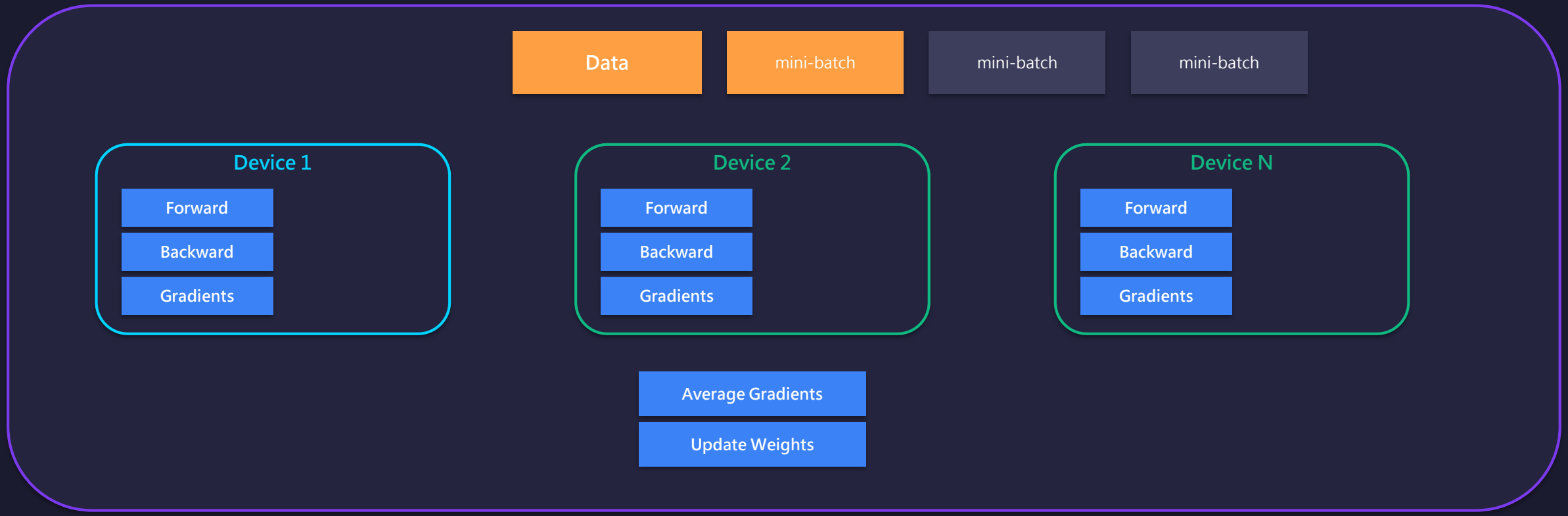
1. When Gradient Accumulation?
2. How Gradient Accumulation efficient?

Gradient Accumulation

每張卡維護自己的梯度，透過集合通訊累積

Each machine maintain self gradient, accumulate gradients using Collective communication.

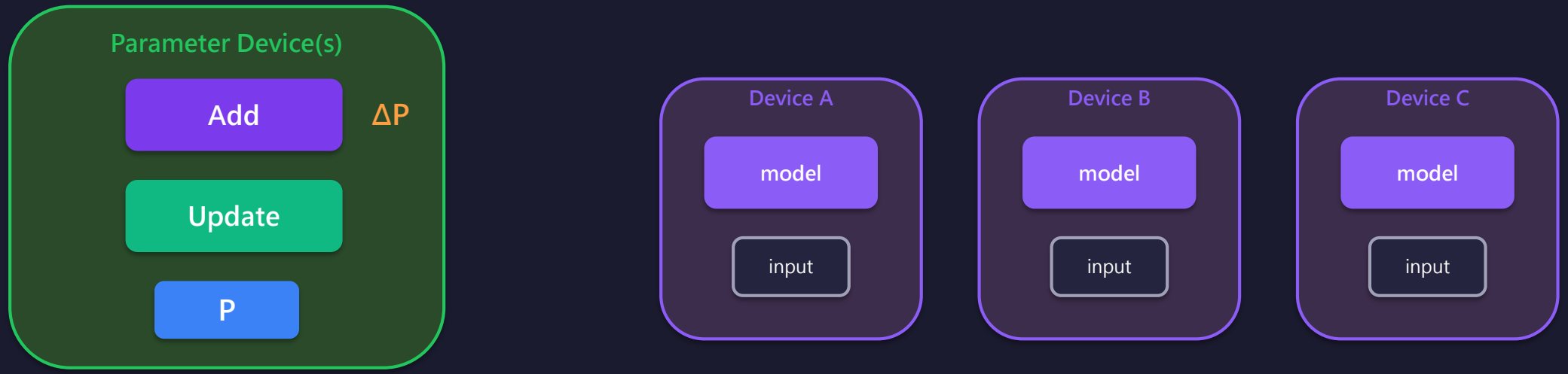
1. When Gradient Accumulation?
2. How Gradient Accumulation efficient?



Gradient Accumulation: Synchronize

同步梯度累積架構

Next round of local computing cannot continue until all working nodes have completed this communication.



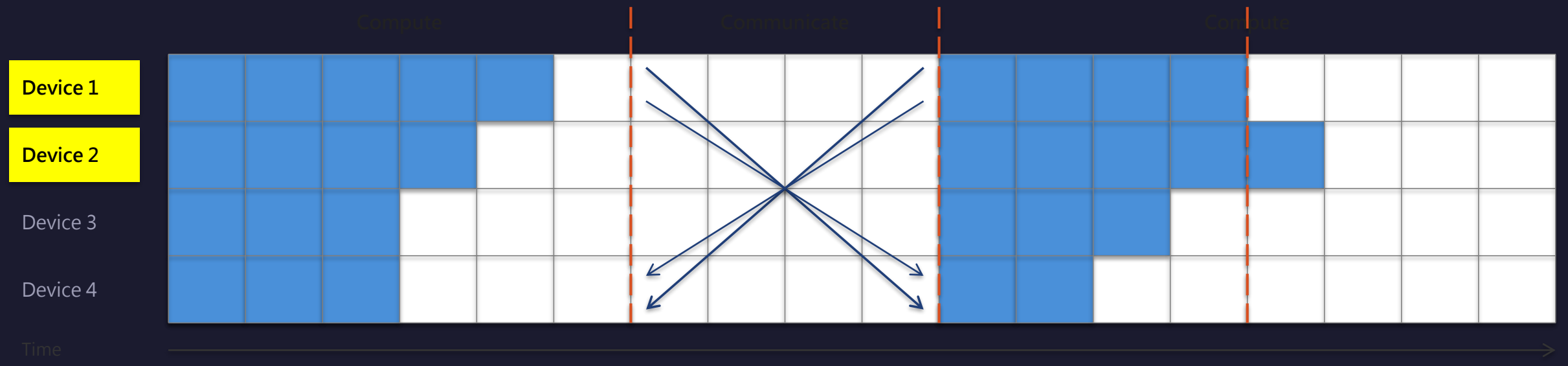
Synchronous Data Parallelism

Gradient Accumulation: Synchronize

優缺點與時序

Advantages: computing and communication are strictly synchronized, ensure parallel execution logic is the same as the serial execution logic.

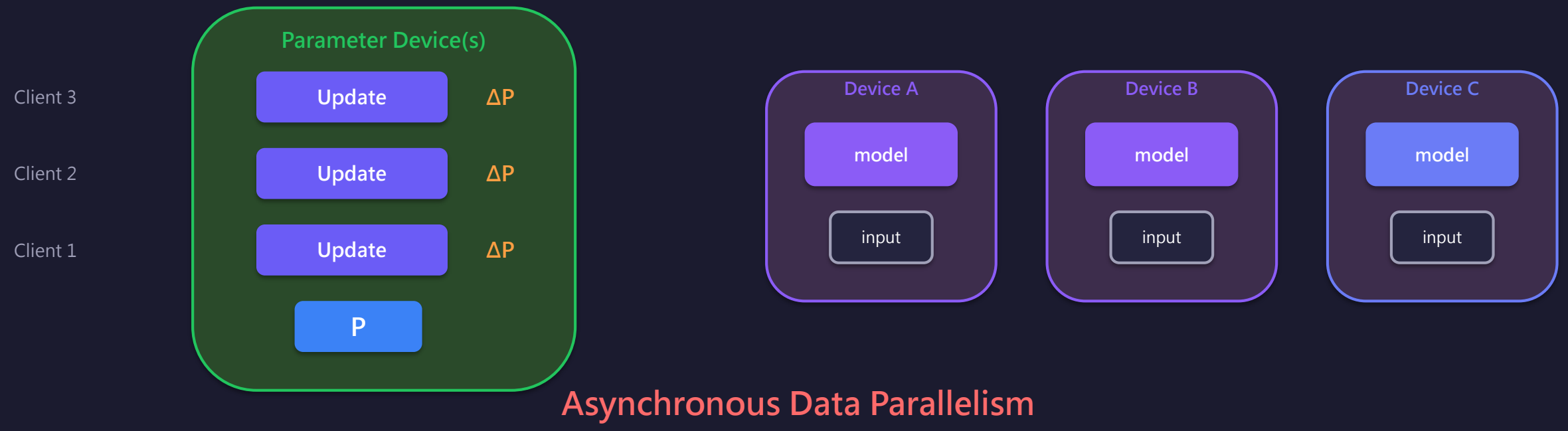
Disadvantages: The working node of the local computing earlier needs to wait for other working nodes to process, which causes a waste of computing hardware.



Gradient Accumulation: Asynchronous

非同步梯度累積架構

After the current batch iteration, communicate with other servers to transmit network model parameters.

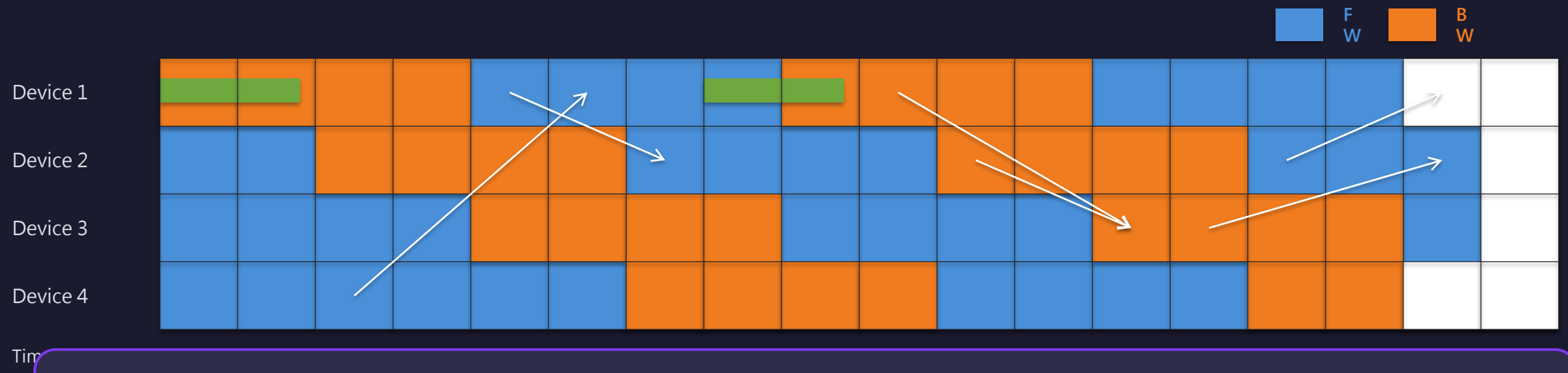


Gradient Accumulation: Asynchronous

優缺點與時序

Advantages: High execution efficiency, no blocking and waiting between communication and execution except for single machine communication time.

Disadvantages: the network model training is not convergent, the training time is long, and the repeated use of model parameters leads to industrialization failure.

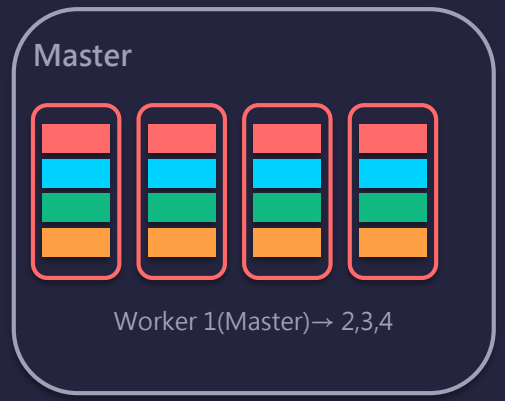


實務上幾乎都用同步方式 — DDP 就是同步 DP
非同步梯度過時導致收斂不穩定，在產業界很少使用

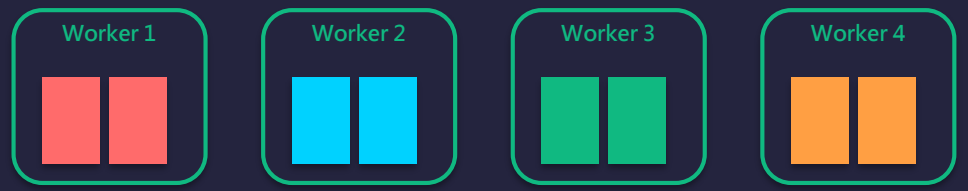
Gradient Accumulation Communication Method

通訊架構比較

GPU0 作為參數伺服器



參數伺服器分散在所有 GPU



Ring All-Reduce：每個 Worker 只跟鄰居通訊
通訊量和 GPU 數量幾乎無關 → 可擴展性好

Distributed Data Parallel, DDP

多行程 + 梯度同步

Adopted multi process parallelism, not restricted by GIL.

Parameters of each process are not synchronized, but the change of parameters.

Using Ring All-Reduce improves the communication efficiency.

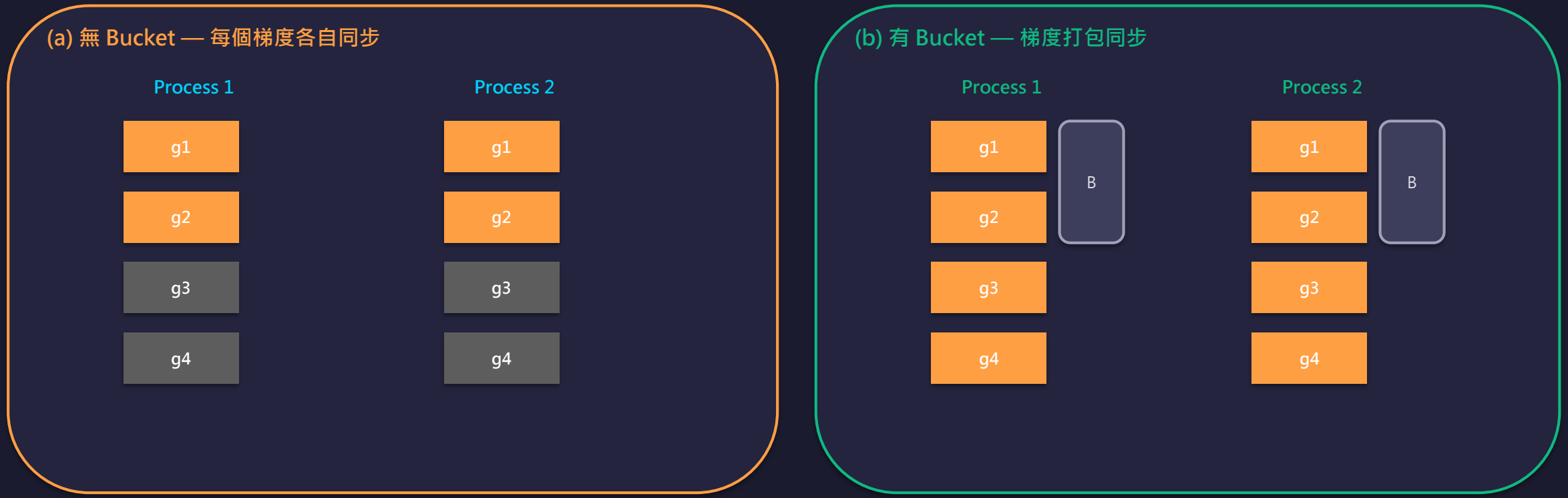


Figure 3: Gradient Synchronization Failures

Distributed Data Parallel, DDP

Gradient Bucket 與通算重疊

1. gradient bucket
2. keep reduce order
3. skip gradient
4. Collective communication

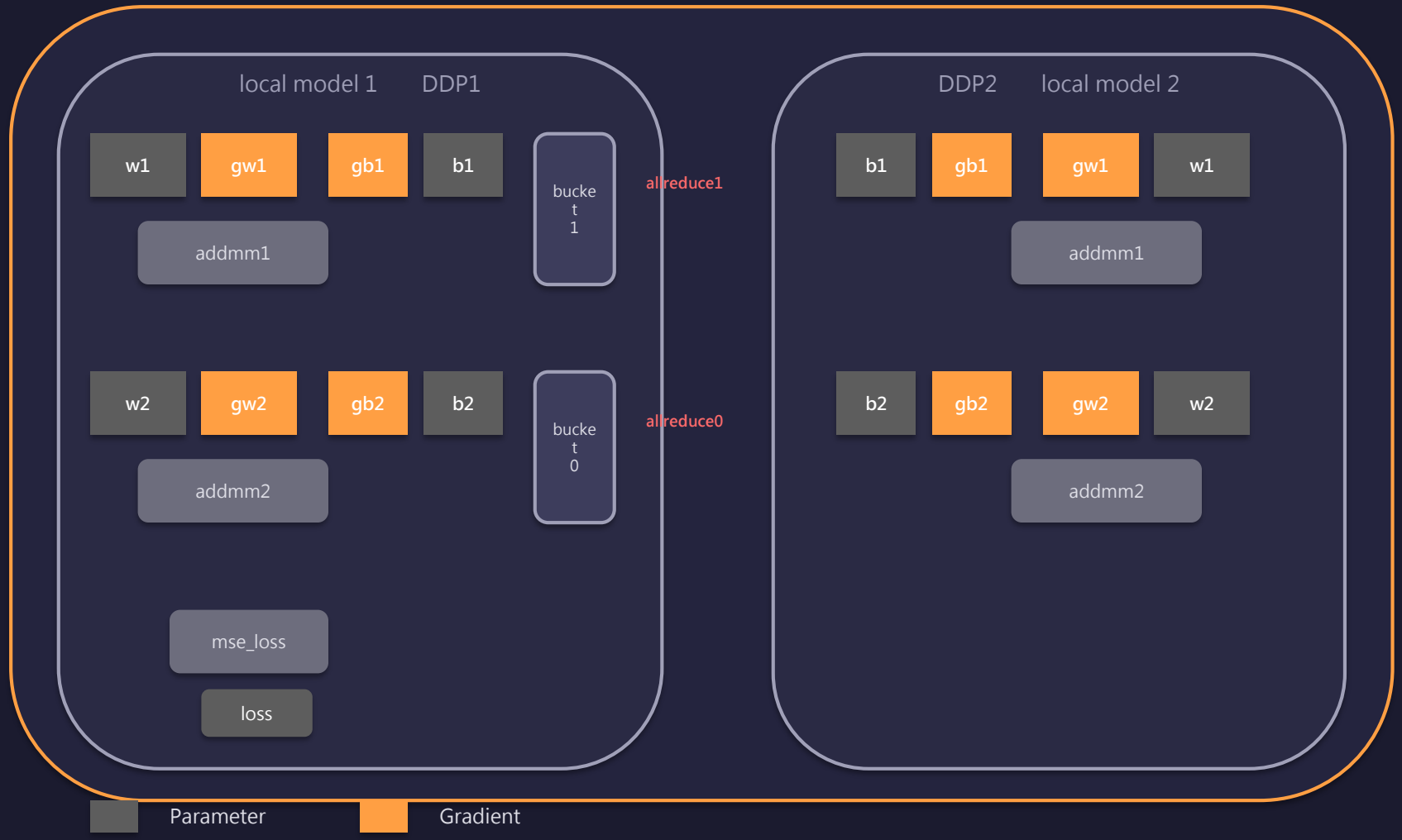
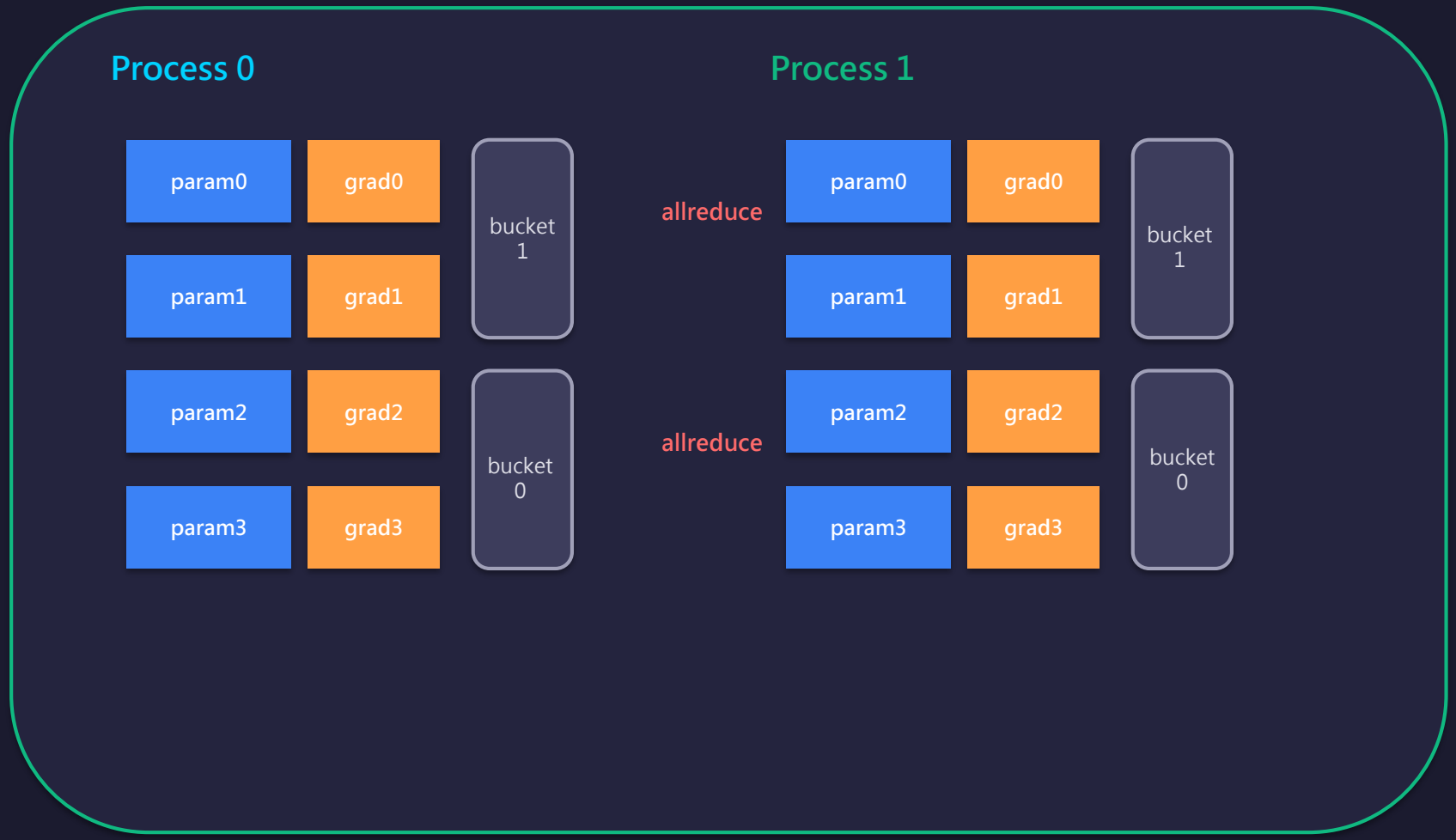


Figure 4: Distributed Gradient Reduction

Distributed Data Parallel, DDP

Gradient Bucket 簡化圖

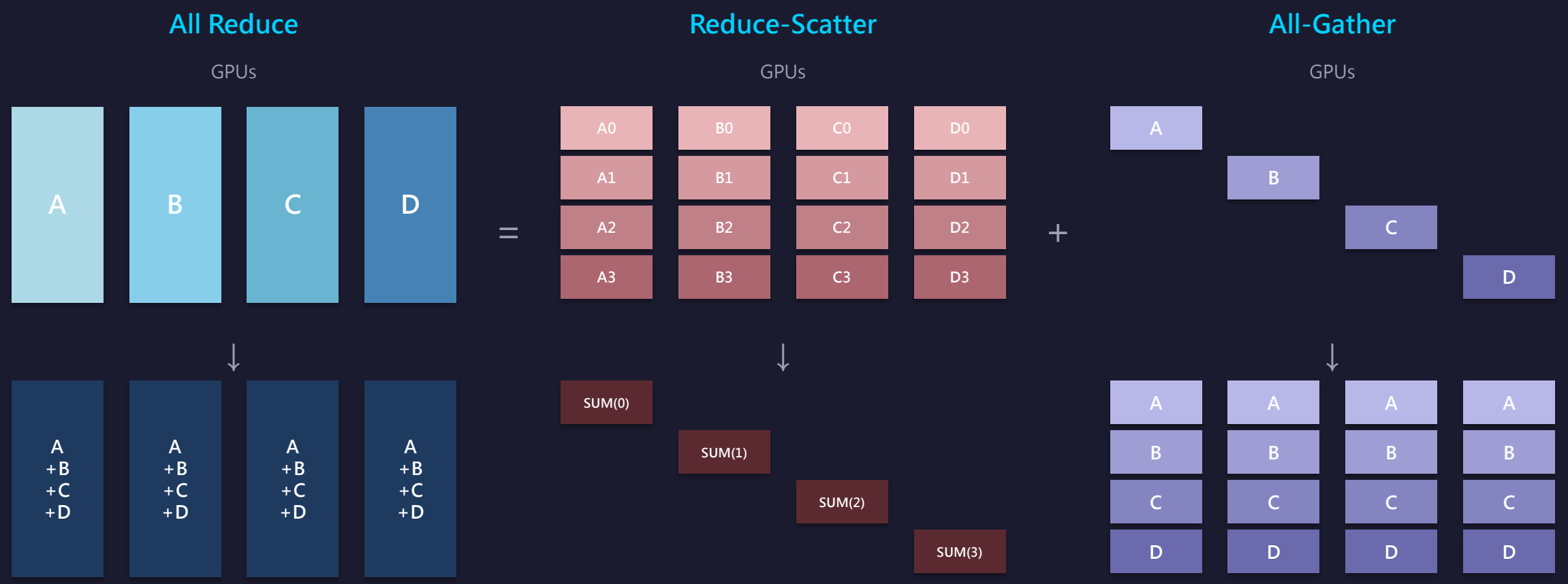
1. gradient bucket
2. keep reduce order
3. skip gradient
4. Collective communication



https://pytorch.org/docs/stable/notes/ddp.html

Collective Communication

All-Reduce = Reduce-Scatter + All-Gather



Model's States

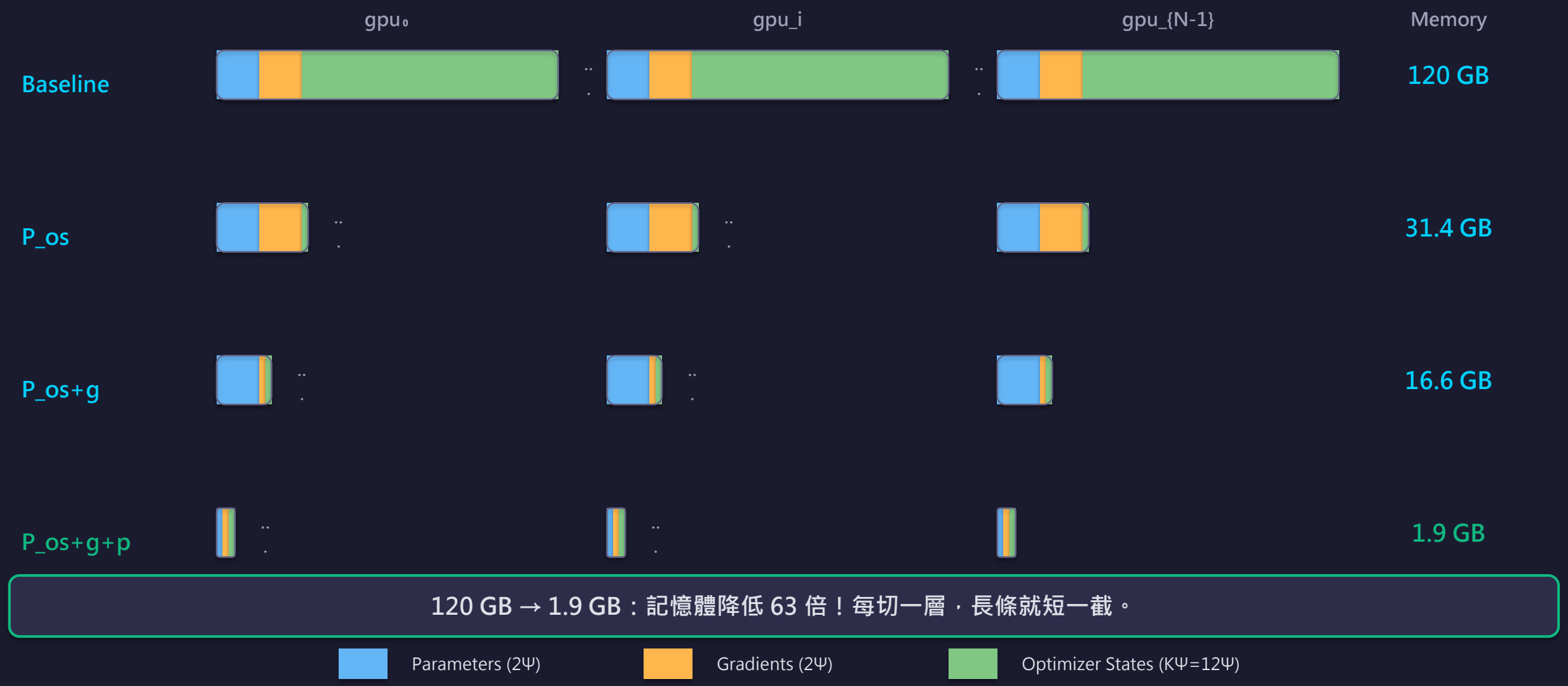
K=12, Ψ=7.5B, Nd=64

階段	gpu_0	gpu_i	gpu_N-1	Memory Consumed	實際用量
Baseline	完整	完整	完整	(2+2+K)*Ψ	120 GB
P_os (ZeRO-1)	P+G完整 1/N OS	P+G完整 1/N OS	P+G完整 1/N OS	2Ψ+2Ψ+K*Ψ/Nd	31.4 GB
P_os+g (ZeRO-2)	P完整 1/N G+OS	P完整 1/N G+OS	P完整 1/N G+OS	2Ψ+(2+K)*Ψ/Nd	16.6 GB
P_os+g+p (ZeRO-3)	1/N 全部	1/N 全部	1/N 全部	(2+2+K)*Ψ/Nd	1.9 GB

Parameters Gradients Optimizer States

ZeRO Memory Optimization

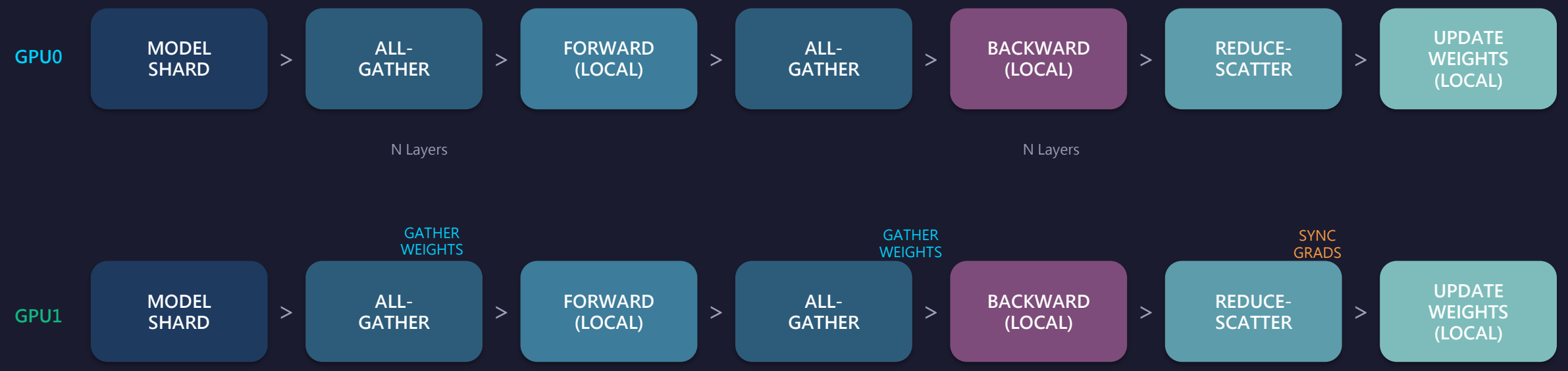
K=12, Ψ=7.5B, Nd=64 — 記憶體視覺化



Fully Sharded Data Parallel, FSDP

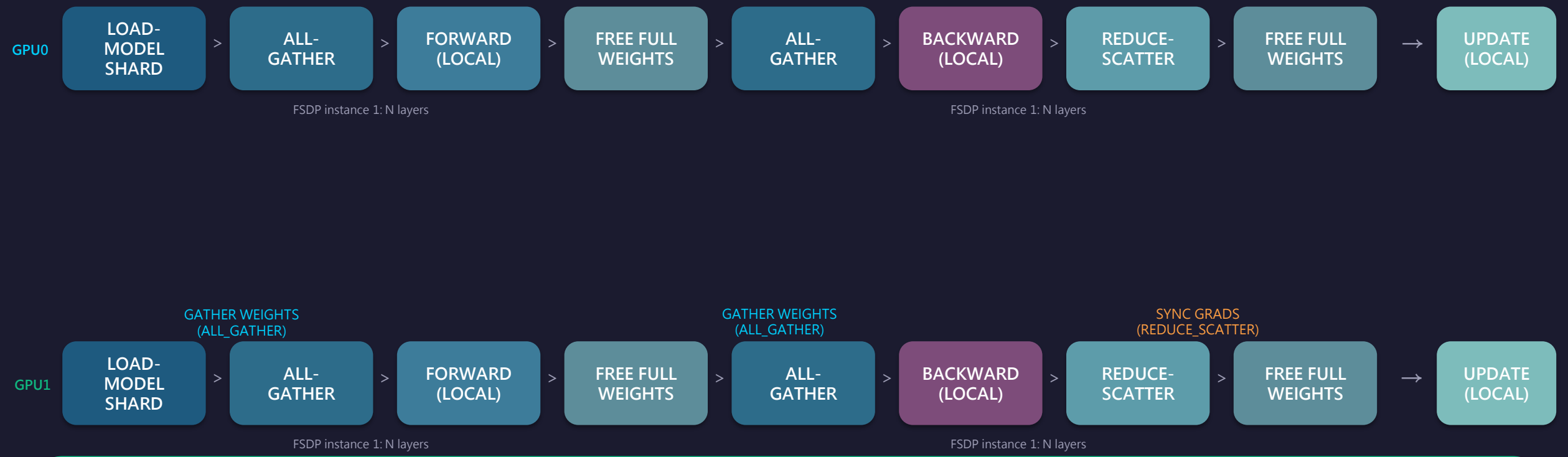
完全分片的資料平行

FSDP shards all of model's parameters, gradients and optimizer states across data-parallel workers and can optionally offload the sharded model parameters to CPUs.



Fully Sharded Data Parallel, FSDP

詳細訓練流程 + CPU Offload

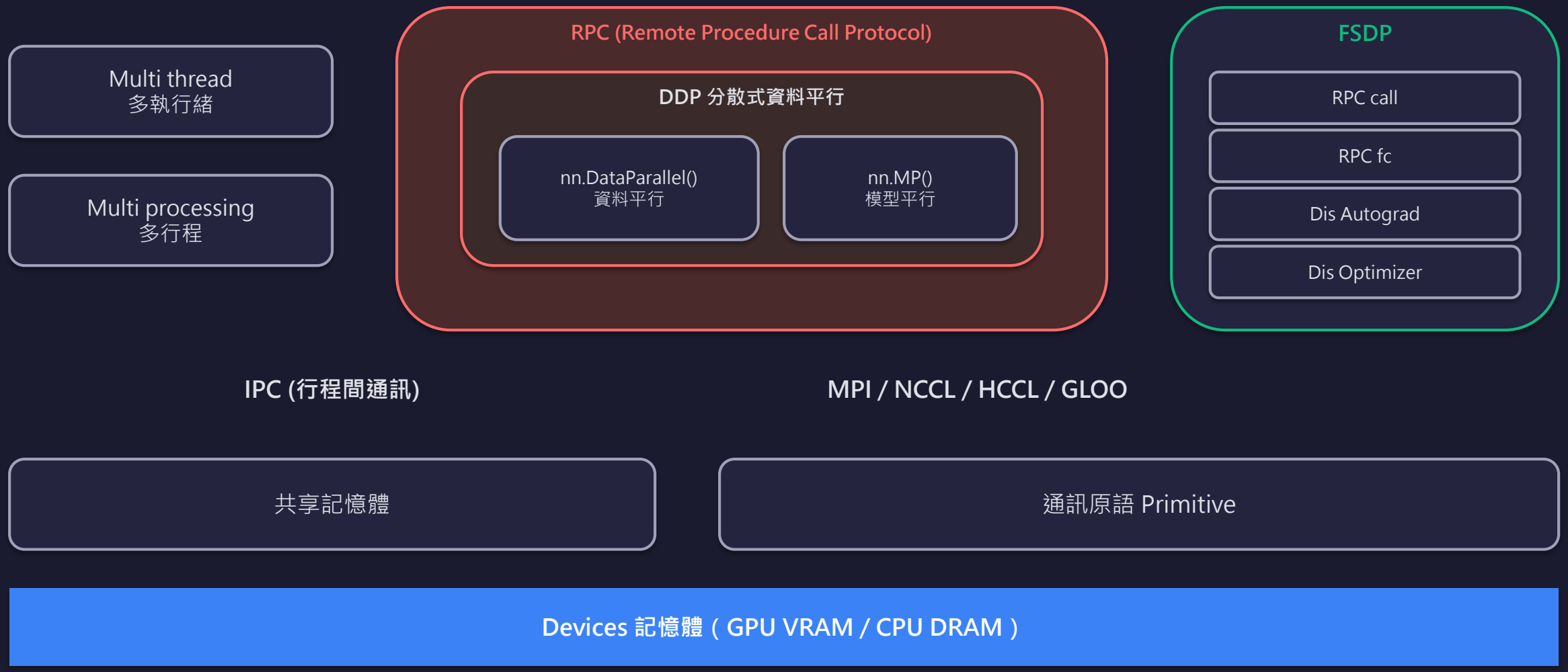


CPU Offload（可選）

Load shard from CPU if CPU Offloaded → 從 CPU 載入分片到 GPU
Offload grads to CPU if CPU offload is enabled → Backward 完把梯度寫回 CPU
CPU 上做 optimizer step → 更省 GPU VRAM，代價是 PCIe 傳輸延遲

PyTorch Implementation

PyTorch 分散式訓練架構



Model's States — 總結回顧

K=12, Ψ=7.5B, Nd=64

階段	gpu_0	gpu_i	gpu_N-1	Memory Consumed	實際用量
Baseline	完整	完整	完整	(2+2+K)*Ψ	120 GB
P_os (ZeRO-1)	P+G完整 1/N OS	P+G完整 1/N OS	P+G完整 1/N OS	2Ψ+2Ψ+K*Ψ/Nd	31.4 GB
P_os+g (ZeRO-2)	P完整 1/N G+OS	P完整 1/N G+OS	P完整 1/N G+OS	2Ψ+(2+K)*Ψ/Nd	16.6 GB
P_os+g+p (ZeRO-3)	1/N 全部	1/N 全部	1/N 全部	(2+2+K)*Ψ/Nd	1.9 GB

Parameters　Gradients　Optimizer States

120 GB → 1.9 GB，降低超過 60 倍！代價是 ZeRO-3 通訊量增加 50%（額外的 All-Gather）

練習作業 2.1

Data Parallel (DP) 的集中式梯度聚合

情境說明

在 DP 架構中，主節點（Master Node, GPU 0）承擔極大的通訊與運算負載。本練習模擬主節點在 Backward Pass 結束後，收集各 Worker 的梯度，計算全域平均值，並執行權重更新。

程式骨架

```
import torch

# 系統初始設定
main_weight = torch.tensor([10.0])  # 初始權重
learning_rate = 0.1

# 模擬 4 個 Worker 計算出的局部梯度
worker_gradients = [
    torch.tensor([2.0]), torch.tensor([4.0]),
    torch.tensor([6.0]), torch.tensor([8.0])
]

# 任務 1: 計算平均梯度 (Gradient Aggregation)
# TODO: 將 worker_gradients 加總並計算平均值
avg_grad = ________________________________

# 任務 2: 執行權重更新 (Weight Update)
# TODO: 新權重 = 舊權重 - (學習率 * 平均梯度)
main_weight = ________________________________

print(f"[主節點] 更新完成，權重: {main_weight.item()}")
```

預期終端機輸出

```
$ python dp_aggregation.py
[主節點] 更新完成，權重: 9.5
```

概念提示

4 個節點的梯度總和為 20.0
平均為 5.0
權重更新：10.0 − (0.1 × 5.0) = 9.5

練習作業 2.2

Distributed Data Parallel (DDP) 去中心化同步

情境說明

DDP 屏除主節點設計，採去中心化架構。本練習模擬兩個 Process 透過 All-Reduce 同步局部梯度，確保各節點獨立更新後模型參數仍保持一致。

程式骨架

```
import os, torch
import torch.distributed as dist
import torch.multiprocessing as mp

def ddp_sync_simulation(rank, world_size):
    os.environ['MASTER_ADDR'] = 'localhost'
    os.environ['MASTER_PORT'] = '12345'
    dist.init_process_group("gloo",
        rank=rank, world_size=world_size)
    my_weight = torch.tensor([10.0])
    my_gradient = torch.tensor(
        [2.0 if rank == 0 else 8.0])
    learning_rate = 0.1

    # 任務 1: All-Reduce 全域梯度同步
    # TODO: dist.all_reduce(my_gradient, ...)
    ________________________________________
    # 任務 2: 計算全域平均梯度
    my_gradient = ______________________
    # 任務 3: 各節點獨立更新權重
    my_weight = ________________________

    print(f"[節點 {rank}] 權重: {my_weight.item()}")
    dist.destroy_process_group()

if __name__ == "__main__":
    mp.spawn(ddp_sync_simulation,
        args=(2,), nprocs=2, join=True)
```

預期終端機輸出

```
$ python ddp_sync.py
[節點 1] 權重: 9.5
[節點 0] 權重: 9.5
```

概念提示

輸出順序可能交錯。
All-Reduce 確保各節點皆取得
梯度總和 10.0，平均為 5.0
→ 權重更新一致：9.5

練習作業 2.3

FSDP 記憶體動態管理

情境說明

FSDP 將模型參數切片（Sharding）突破單一設備記憶體瓶頸。本練習驗證：靜態時僅保留切片；Forward Pass 前動態 All-Gather 收集完整參數；運算後立即釋放，避免 OOM。

程式骨架

```
import os, torch
import torch.distributed as dist
import torch.multiprocessing as mp

def fsdp_memory_mgmt(rank, ws):
    os.environ['MASTER_ADDR'] = 'localhost'
    os.environ['MASTER_PORT'] = '12345'
    dist.init_process_group("gloo",
        rank=rank, world_size=ws)
    # 1. 靜態存儲：僅保管專屬切片
    my_shard = torch.tensor([float(rank+1)])
    print(f"[節點 {rank}] [靜態] {my_shard.tolist()}")
    # 2. All-Gather 收集全域參數
    full_list = [torch.zeros(1)
                 for _ in range(ws)]
    # TODO: dist.all_gather(full_list, my_shard)
    ________________________________________
    # TODO: 用 torch.cat 組合 full_weight
    full_weight = _______________________
    print(f"[節點 {rank}] [配置] {full_weight.tolist()}")
    # --- 模擬 Forward/Backward ---
    print(f"[節點 {rank}] [運算] 完成")
    # 3. 記憶體釋放
    # TODO: del full_weight
    ________________________________________
    try:
        print(full_weight)
    except NameError:
        print(f"[節點 {rank}] [回收] 已釋放")
    dist.destroy_process_group()

if __name__ == "__main__":
    mp.spawn(fsdp_memory_mgmt,
        args=(2,), nprocs=2, join=True)
```

預期終端機輸出

```
$ python fsdp_memory.py
[節點 0] [靜態] 切片: [1.0]
[節點 1] [靜態] 切片: [2.0]
[節點 0] [配置] 完整: [1.0, 2.0]
[節點 0] [運算] 完成
[節點 0] [回收] 記憶體已釋放

[節點 1] [配置] 完整: [1.0, 2.0]
[節點 1] [運算] 完成
[節點 1] [回收] 記憶體已釋放
```

Part 4

Tensor Parallelism

張量平行 — 切開層內的權重矩陣

訓練加速技術全景

Training Acceleration Landscape — 你的錢 vs 你的時間

你的時間 ↑

Gradient Checkpointing

省 VRAM、用時間換空間
少存 activation
backward 時重算

Gradient Accumulation

等效大 batch
多次小 batch 累積梯度
不需額外 GPU

Mixed Precision

FP16 / BF16 訓練
速度 2x+、省 VRAM
幾乎必開的基本優化

Distributed Training

多 GPU 平行
DP / DDP / FSDP
花錢換速度

4D Parallel + LAMB

TP + PP + SP + DP
超大規模 training
大 batch optimizer

省錢用技巧 ←————————————→ 花錢買硬體

你的錢 →

Model Parallelism, MP 模型平行

模型平行的兩大策略

Tensor Parallelism 張量平行

- **Principles 平行原理** — 張量如何切分到多個裝置
- **MatMul 矩陣乘算子平行** — 列切分 (Column) vs 行切分 (Row)
- **Loss 損失平行** — 大詞彙表的分散式 Cross-Entropy
- **Transformer 算子平行** — MLP 列→行 + Self-Attention 頭切分
- **Tensor Redistribution** — 張量重排（MindSpore / DTensor）
- **Stochastic Control** — 隨機性控制（Dropout seed 同步）

Pipeline Parallelism 管線平行

- **層間切分**
  按 Layer 分配到不同 GPU
- **Pipeline Bubble**
  GPU 閒置問題與排程優化
- **1F1B 排程**
  One-Forward-One-Backward
- **Interleaved PP**
  交錯排程減少氣泡

從 Data Parallelism 到 Model Parallelism

我們已學完 DP，接下來進入模型平行

已學內容回顧：

DP

nn.DataParallel
多執行緒、受 GIL 限制

DDP

多行程 + Gradient Bucket
通算重疊 + Ring All-Reduce

FSDP / ZeRO

參數+梯度+Optimizer 全分片
120GB → 1.9GB

▼ DP 前提：模型放得進單張 GPU ▼

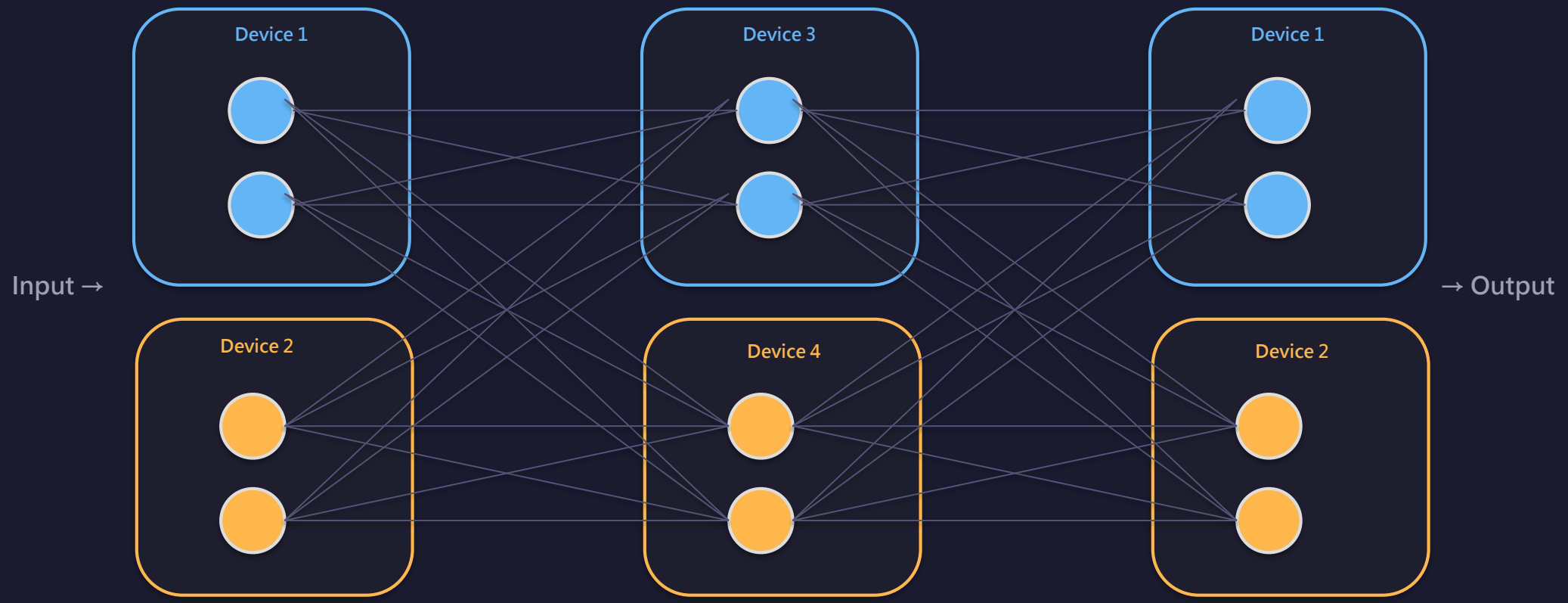
如果模型太大，一張 GPU 放不下呢？

→ 模型平行 (Model Parallelism)：把模型本身切開
- Pipeline Parallelism (PP)：按「層」切分 → 層間平行
- Tensor Parallelism (TP)：按「層內參數」切分 → 層內平行

MP(II): Tensor Parallelism 張量平行

將層內參數切分到不同裝置，即層內平行

張量平行（層內平行）：每個裝置負責每一層的部分神經元



TP 特點：每層都需要 All-Reduce / All-Gather 通訊 → 適合高頻寬互連（如 NVLink）
優勢：無 Pipeline Bubble，記憶體隨 TP degree 線性下降

Tensor Parallelism 核心問題

如何切分？如何保證數學正確性？

Pipeline Parallelism (PP)

- 按模型 Layer 切分到不同裝置
- 層間平行 — 每個裝置完整負責某幾層
- 通訊：只需 P2P（Activation 傳遞）
- 問題：Pipeline Bubble（GPU 閒置）

Tensor Parallelism (TP)

- 將層內的參數切分到不同裝置
- 層內平行 — 每個裝置只算層的一部分
- 通訊：每層需要 All-Reduce / All-Gather
- 適合高頻寬互連（NVLink 節點內）

Tensor Parallelism 要解決兩個核心問題：

Q1

如何切分？（Segmentation Mode）

矩陣可以按行切分、按列切分、或複製 — 選擇不同切法會影響通訊模式

Q2

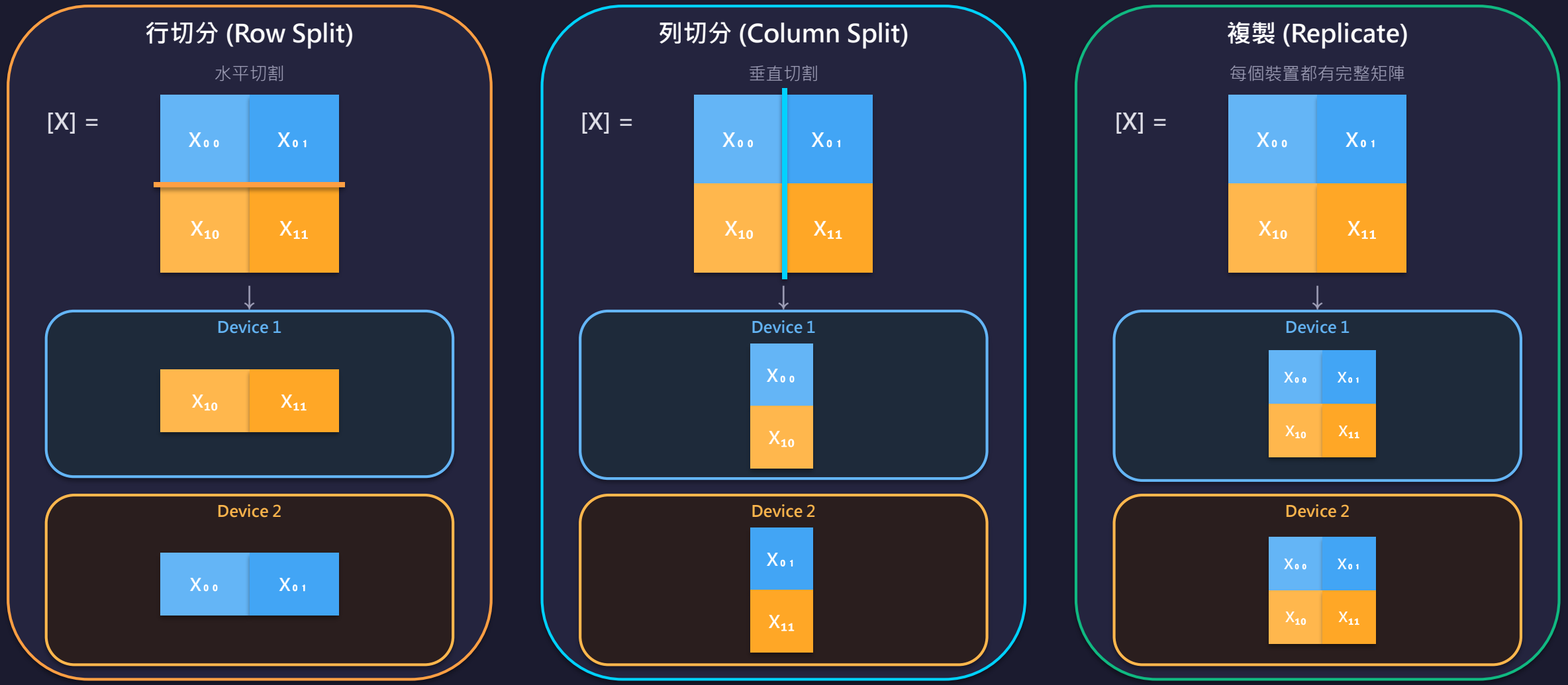
如何保證數學正確性？（Mathematical Consistency）

切分後的分散式計算結果必須等於原始單卡計算結果 — 需要搭配適當的集合通訊

Mathematical Principles 數學原理

張量切分方式：行切分 / 列切分 / 複製

張量切分方式 — 以 2×2 矩陣 [X] 為例



MatMul 矩陣乘算子平行 — 列切分

A 按列切分：$X \times [A_1 \ A_2] = [Y_1 \ Y_2] \rightarrow$ All-Gather $\rightarrow Y$

$XA = Y$ （X 為啟動輸入，A 為算子權重）

將 A 按列切分：$X \times [A_1 \ A_2] = [XA_1 \ XA_2] = [Y_1 \ Y_2] = Y$

每個裝置各算出 Y 的一部分，最後用 All-Gather 拼回完整 Y

列切分

Device 1: $X \times A_1 = Y_1$

Device 2: $X \times A_2 = Y_2$

列切分：X 需複製到所有裝置（Broadcast/Replicate），結果 Y 分片 → All-Gather 合併
通訊量：All-Gather 一次（收集 Y 的分片）

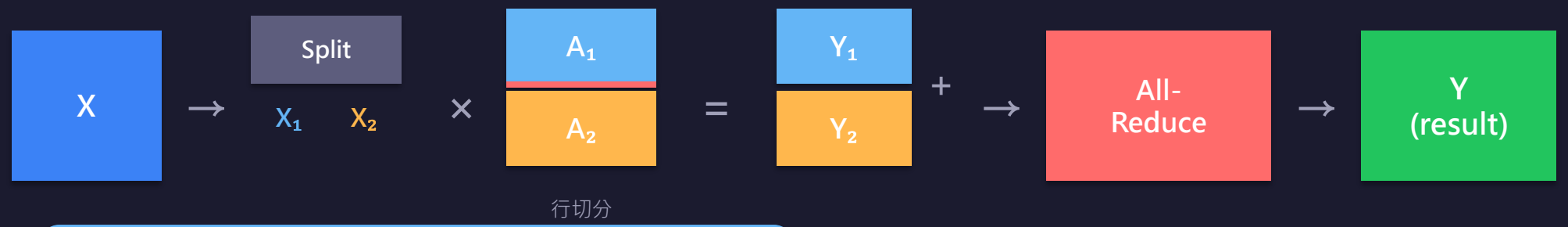
MatMul 矩陣乘算子平行 — 行切分

A 按行切分：需要同時切分 X，結果用 All-Reduce 求和

$XA = Y$ （X 為啟動輸入，A 為算子權重）

將 A 按行切分，X 也必須按列切分（配合矩陣乘法維度）：

$$[X_1 \;\; X_2] \times [A_1; A_2] = X_1A_1 + X_2A_2 = Y$$



Device 1: $X_1 \times A_1 = Y_1$ (partial)

Device 2: $X_2 \times A_2 = Y_2$ (partial)

行切分：X 需一起切分（Split），每個裝置算出部分和 → All-Reduce 加總得到完整 Y
通訊量：All-Reduce 一次（加總 Y_1+Y_2），但 X 需要先做切分（Split / Scatter）

MatMul 切分小結：列切分 vs 行切分

Column Partition vs Row Partition — 通訊模式不同

	列切分 (Column)	行切分 (Row)
權重 A 切法	A 按列拆 $[A_1 \mid A_2]$	A 按行拆 $[A_1 ; A_2]$
輸入 X	X 複製（不切）	X 也要切 $[X_1 \mid X_2]$
各裝置計算	$XA_1 = Y_1,\ XA_2 = Y_2$	$X_1A_1 = Y_1,\ X_2A_2 = Y_2$
結果 Y	Y 按列分片	Y_1, Y_2 是部分和
還原通訊	All-Gather（拼接）	All-Reduce（加總）
輸入通訊	Broadcast / 複製 X	Split / Scatter X

Megatron-LM 的 TP 策略：MLP 先列切分再行切分，串接後只需 2 次 All-Reduce（不需 All-Gather）

Transformer: MLP 算子平行 (I)

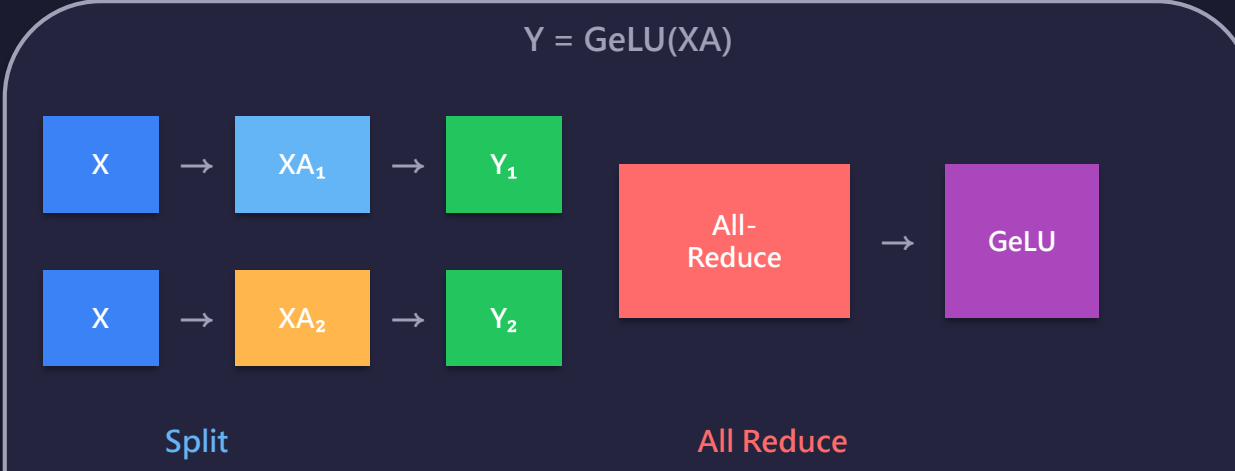
Option 1 — A 行切分 + GeLU：$Y = GeLU(X_1A_1 + X_2A_2)$

Option 1: A 行切分

$$X = [X_1, X_2], \quad A = [A_1 ; A_2]$$

$$Y = GeLU(X_1A_1 + X_2A_2)$$

→ 先 All-Reduce 再 GeLU（非線性在通訊之後）



⚠ Option 1 的問題

- GeLU 是非線性函數：$GeLU(X_1A_1 + X_2A_2) \neq GeLU(X_1A_1) + GeLU(X_2A_2)$
- 必須先做 All-Reduce 把部分和加起來，再過 GeLU
- 問題：All-Reduce 在 GeLU 前面 → 阻塞計算管線，通訊開銷無法隱藏
- 結論：這種方式在實作中不是最佳選擇

Transformer: MLP 算子平行 (II)

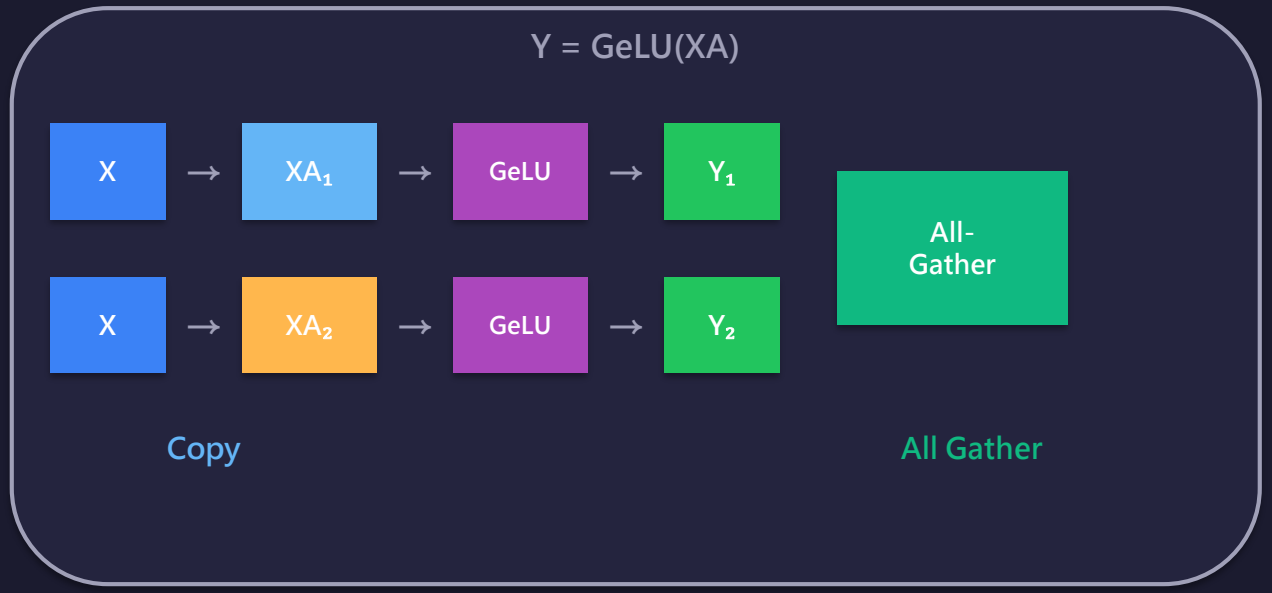
Option 2 — A 列切分 + GeLU：$[Y_1,Y_2] = [GeLU(XA_1), GeLU(XA_2)]$

Option 2: A 列切分 ✓（Megatron 採用）

$A = [A_1 \mid A_2]$ （列切分）

$$[Y_1, Y_2] = [GeLU(XA_1), GeLU(XA_2)]$$

→ 先 GeLU 再通訊（非線性在通訊之前）



✓ Option 2 的優勢

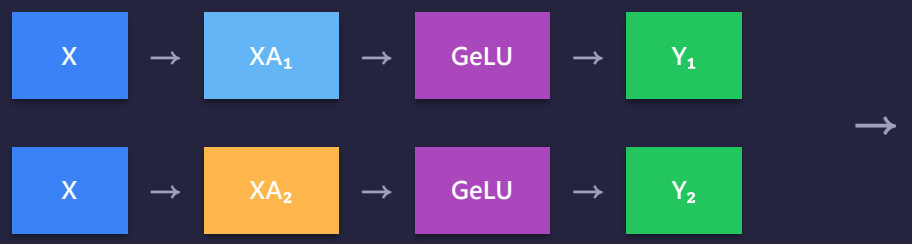
- GeLU 是逐元素操作 (element-wise)：可以先各自做 GeLU 再合併
- $GeLU(XA_1)$ 和 $GeLU(XA_2)$ 可以獨立在各裝置上計算
- 通訊放到非線性之後 → 計算與通訊更容易重疊
- Megatron-LM 採用此方案，搭配第二層 B 的行切分消除 All-Gather

Transformer: MLP 完整 TP 架構

$Z = \text{Dropout}(\text{GeLU}(XA) \cdot B)$ — 列切分 A + 行切分 B → 只要一次 All-Reduce

MLP 完整流程：第一層 A 列切分 + 第二層 B 行切分

Stage 1: $Y = \text{GeLU}(XA)$ A 列切分



Copy (f)

Stage 2: $Z = \text{Dropout}(YB)$ B 行切分

All-Reduce (g)

Megatron-LM MLP TP 精妙設計

- A 列切分 (Column) → GeLU 可各自做 → 不需先 All-Reduce
- B 行切分 (Row) → $Y_1B_1 + Y_2B_2 = YB$ → All-Reduce 加總一次即可
- 整個 MLP 只需 1 次 Copy（前向 f）+ 1 次 All-Reduce（後向 g）
- 相比 naive 做法省掉 All-Gather，通訊量降一半

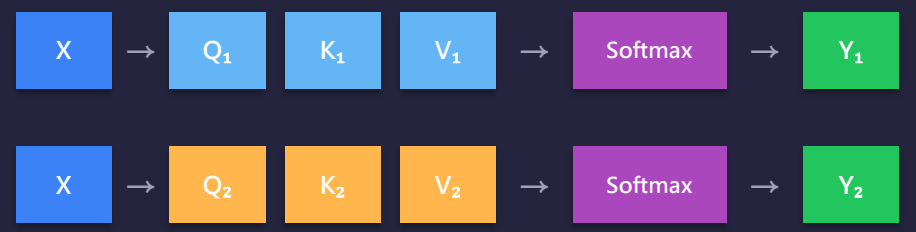
Transformer: Self-Attention 算子平行

$Z = \text{Dropout}(\text{Self-Attention}(X) \cdot B)$ — 按 Attention Head 切分

Multi-Head Attention 天然適合 TP：每個 Head 獨立計算

$Q = [Q_1, Q_2],\ K = [K_1, K_2],\ V = [V_1, V_2]$ — 按 Head 列切分 Q/K/V 投影矩陣

$Y = \text{Self-Attention}(X)$ Head 切分



Copy (f)

$Z = \text{Dropout}(YB)$ B 行切分

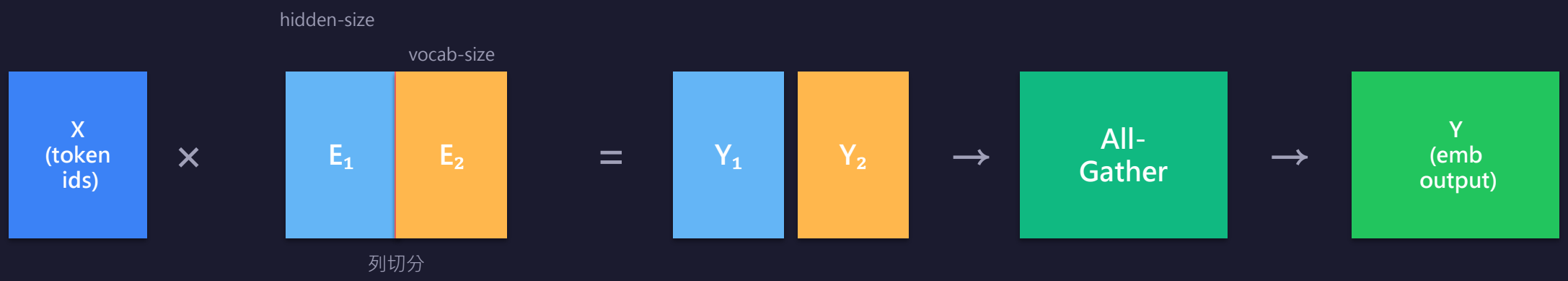
All-Reduce (g)

- Self-Attention 按 Head 切分：每個裝置獨立算 Softmax(Q_iK_i^T/√d)V_i
- Softmax 內部 Head 獨立 → 不需跨裝置通訊
- 投影矩陣 B 行切分 → 和 MLP 一樣 All-Reduce 一次
- 一個 Transformer Block = MLP (1 All-Reduce) + Attention (1 All-Reduce) = 共 2 次 All-Reduce

Transformer: Embedding 算子平行

Embedding 表按列切分 → All-Gather 合併

以 GPT-2 為例：vocab_size = 50,257，Embedding 表 [vocab_size × hidden_size]
按照 vocab 維度（列）切分，每張卡只存一半的詞向量表 → 查表後 All-Gather 合併



Embedding TP 策略

- 語言模型 Embedding 表很大：GPT-2: 50,257 × 1,600 ≈ 305MB (FP32)
- 按 vocab 維度列切分：每張卡存 [vocab_size/n × hidden_size]
- 查表後各卡得到部分向量 → All-Gather 拼接成完整 Embedding 輸出
- 推薦系統場景還可以用 Table-wise（按表分配）vs Column-wise（按列分配）

Cross Entropy Loss 平行

大詞彙表模型的分散式損失計算

問題：Logits 矩陣太大，單卡放不下

- 語言模型最後一層輸出 logits 維度: [batch_size × seq_len, vocab_size]
- LLaMA: vocab = 32,000+；GPT-4 等級: vocab > 100K
- 一般做法：softmax + cross_entropy → 需要完整 logits 和 labels
- 當類別數 (vocab_size) 太大時，單卡記憶體不足以儲存完整 logit 矩陣

Cross Entropy Loss 公式

二分類：

$$L = -1/N \Sigma [y_i \cdot \log(p_i) + (1-y_i) \cdot \log(1-p_i)]$$

多分類（語言模型）：

$$L = -1/N \Sigma_i \Sigma c\, y_i c \cdot \log(p_i c)$$

分散式 Cross Entropy 的四步流程

Step 1: 數據拆分

logits 按 vocab 維度切分 + labels one-hot scatter

Step 2: 最大值同步

All-Reduce(Max) 取全局最大值防止溢出

Step 3: exp sum 計算

各卡局部 exp sum → All-Reduce(Sum) 取全局分母

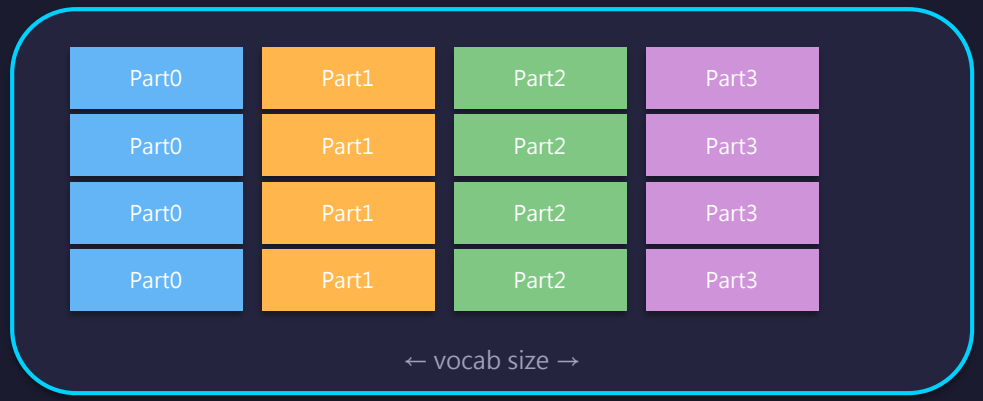
Step 4: 計算 Loss

logits × one_hot → All-Reduce(Sum) → log softmax → loss

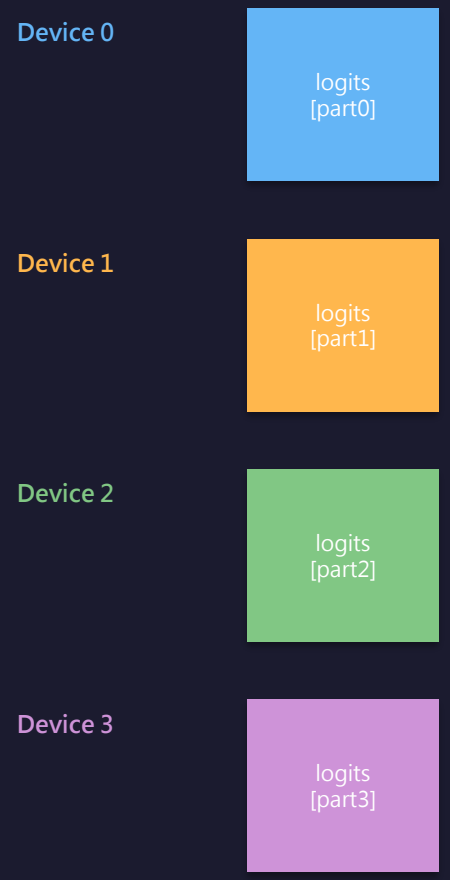
Cross Entropy Loss — Step 1: 數據拆分

logits 按 vocab 維度切分，labels 做 one-hot 後 scatter

Logits [bs×seq, vocab] 按 vocab 維度切分



→



Labels 做 one-hot 後 scatter

1. labels [bs×seq] → one-hot [bs×seq, vocab]
2. one-hot 矩陣也按 vocab 維度切分
3. scatter 到各裝置

每個裝置得到:
- logits_part [bs×seq, vocab/n]
- labels_part [bs×seq, vocab/n]

- 每張卡只需存 vocab/n 大小的 logits 子矩陣 → 記憶體降為 1/n
- Labels 的 one-hot 大部分為 0，只有正確類別位置非零
- Scatter 後每張卡的 labels 仍保持 one-hot 格式（可能全為 0）

Cross Entropy Loss — Step 2-3

最大值同步 + exp sum 計算

Step 2: 最大值同步

- Softmax 需要減去最大值防止 exp 溢出
- 各卡先求局部最大值 → All-Reduce(Max)
- 取得全局最大值 x_max

$$x_max = MAX_p(\ MAX_k(x_k)\ )$$

通訊：All-Reduce (Max) 一次

Step 3: exp sum 計算

- 各卡算局部 exp sum (扣掉 x_max)
- exp_sum_local = Σ_k exp(x_k - x_max)
- All-Reduce(Sum) → 取得全局 exp_sum

$$softmax(x_i) = exp(x_i - x_max) / \Sigma_p\ \Sigma_k\ exp(x_k - x_max)$$

通訊：All-Reduce (Sum) 一次

Step 2-3 各裝置流程

Cross Entropy Loss — Step 4: 計算 Loss

logits × one_hot → All-Reduce(Sum) → log softmax → loss

Step 4: 計算最終損失值

各卡 logits_part × labels_one_hot_part → 得到 label 位置的 logit 值
All-Reduce(Sum) 同步 → log softmax → 加負號得到 cross entropy loss

$$\text{loss} = -\log(\exp(x_target - x_max) / global_exp_sum)$$
$$= -(x_target - x_max) + \log(global_exp_sum)$$

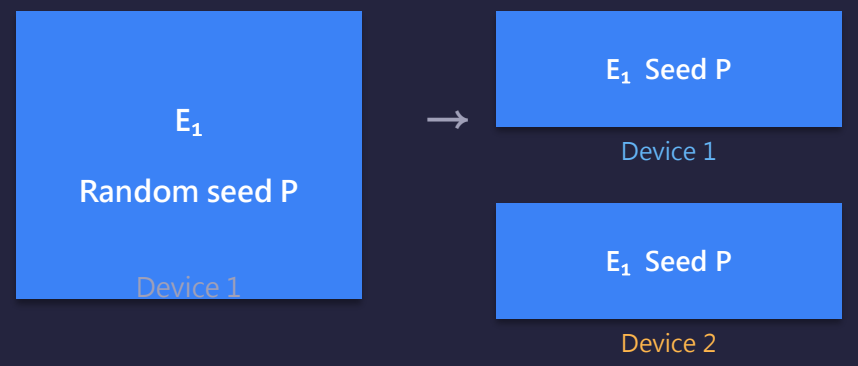
分散式 CE Loss 總通訊量：Step2 All-Reduce(Max) + Step3 All-Reduce(Sum) + Step4 All-Reduce(Sum) = 3 次輕量通訊

Stochastic Control (I): 參數初始化隨機性

TP 切分後的參數初始化必須使用不同的隨機種子

✕ 錯誤做法：相同 Seed

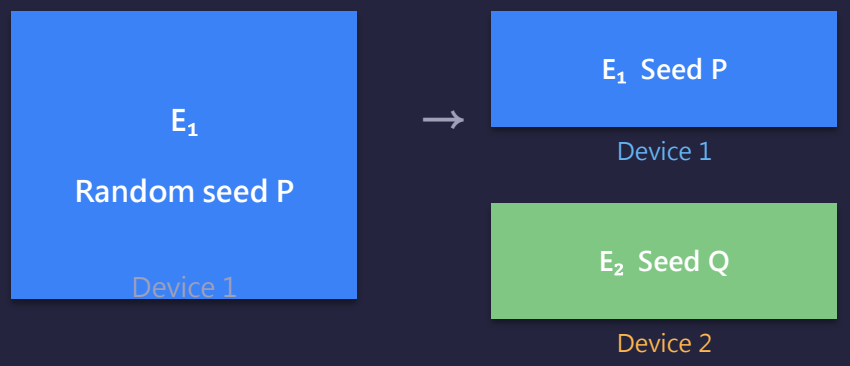
參數初始化後兩個設備的參數
將會初始化為相同的數值
→ 與單設備上的參數在數學上不等價



完全相同！✕

✓ 正確做法：不同 Seed

將參數切分到多個卡上後
修改相應卡的隨機性
→ 保證各個卡的隨機種子不同



不同種子 ✓

- TP 切分後，每個分片應該有不同的初始化值（等價於原始大矩陣的不同部分）
- 如果都用相同 seed → 所有分片值相同 → 失去真正隨機性 → 訓練會退化
- 正確做法：seed_per_device = base_seed + tp_rank，確保每個分片獨立初始化

Stochastic Control (II): Dropout 隨機性

TP 中 Dropout 的隨機種子需要區分「該同步」vs「該不同」

需要相同 Seed 的情況

- 輸入 X 在各裝置上是複製 (Copy) 的
- X 過 Dropout 時，各裝置的結果必須一致
- 否則 $X_1 \neq X_2$ → 後續計算不再等價

例如：Embedding 層 Dropout
→ 各卡使用相同 Dropout Seed

需要不同 Seed 的情況

- 參數或激活值已經被 TP 切分
- 各裝置上是不同的分片
- Dropout mask 應該也不同

例如：MLP 內部 Dropout
→ 各卡使用不同 Dropout Seed

場景	Dropout Seed	原因
複製的輸入 X (Copy)	相同 Seed	保持輸入一致性
TP 切分後的激活值	不同 Seed	各分片獨立操作
All-Reduce 前的局部結果	不同 Seed	加總後自然隨機
LayerNorm / 其他全量操作	相同 Seed	全量操作需一致

Tensor Parallelism 總結

Summary — 張量平行核心要點

1

模型平行 = TP + PP

模型平行分為張量平行和流水線平行。張量平行主要是層內平行，流水線主要是層間平行。
一般來說：節點內使用 TP（NVLink 高頻寬），節點間使用 DP。

2

切分方式：Row / Column / Replicate

張量平行主要是對數據（張量）進行切分，切分方式有行 (Row) 切分和列 (Col) 切分。
而通過複製 (Replicate) 組合可以形成多種通訊形式：All-Reduce / All-Gather。

3

MatMul 拓展到 MLP / Attention / Embedding

張量平行最常見的是 MatMul 算子平行。通過 MatMul 可以拓展到：
Embedding（列切分）、MLP（列→行）、Transformer Self-Attention（按 Head 切分）等算子平行。

4

注意隨機性控制

張量平行的時候值得注意的是隨機性問題，需要注意帶有隨機性算子的隨機種子設置：
參數初始化需要不同 Seed，Dropout 需要根據場景選擇相同或不同 Seed。

練習作業 3.1

矩陣乘法 — 列切分 (Column Partition) 驗證

情境說明

TP 的核心是將矩陣乘法拆分到多個裝置。本題驗證列切分：將權重 A 按列切成兩份，各自與完整 X 相乘，最後用 All-Gather（torch.cat）拼接出完整結果。

程式骨架

```
import torch
torch.manual_seed(42)
X = torch.randn(4, 8)  # 輸入
A = torch.randn(8, 6)  # 權重
Y_ref = X @ A          # 標準答案

# 任務 1: 將 A 按列切分成兩片 (各 8×3)
A1, A2 = ______________________________

# 任務 2: 各「裝置」用完整 X 乘分片
Y1 = ___________________________________
Y2 = ___________________________________

# 任務 3: All-Gather — 按列拼接還原
Y_tp = _________________________________

assert torch.allclose(Y_ref, Y_tp)
print("✓ 列切分驗證通過！")
```

預期輸出

✓ 列切分驗證通過！

概念提示

A 列切分 → X 不用切 (Replicate)

Y_1, Y_2 是 Y 的左右兩半
→ cat 拼接 = All-Gather

這是 Megatron MLP 第一層
採用的切分方式

練習作業 3.2

矩陣乘法 — 行切分 (Row Partition) 驗證

情境說明

行切分與列切分不同：權重 A 按行切、輸入 X 也必須按列切，各裝置算出「部分和」，需要 All-Reduce（加總）才能還原完整結果。

程式骨架

```
import torch
torch.manual_seed(42)
X = torch.randn(4, 8)
A = torch.randn(8, 6)
Y_ref = X @ A

# 任務 1: A 按行切分 (各 4×6)
A1, A2 = ______________________________

# 任務 2: X 按列切分 (各 4×4)
X1, X2 = ______________________________

# 任務 3: 各「裝置」計算部分和
Z1 = ___________________________________
Z2 = ___________________________________

# 任務 4: All-Reduce (Sum) 還原
Y_tp = _________________________________

assert torch.allclose(Y_ref, Y_tp)
print("✓ 行切分驗證通過！")
```

預期輸出

✓ 行切分驗證通過！

概念提示

A 行切分 → X 也必須按列切
（維度要對齊！）

Z_1 和 Z_2 形狀相同（4×6）
是部分和 → 加總 = All-Reduce

這是 Megatron MLP 第二層
採用的切分方式

練習作業 3.3

Megatron-LM MLP 完整 TP 模擬

情境說明

Megatron MLP：第一層 A 列切分 + GeLU + 第二層 B 行切分 + All-Reduce。驗證分散式結果與單卡計算完全一致，整個 MLP 只需 1 次 All-Reduce。

程式骨架

```
import torch
import torch.nn.functional as F
torch.manual_seed(42)
X = torch.randn(4, 8)
A = torch.randn(8, 12)  # 第一層
B = torch.randn(12, 6)  # 第二層
Y_ref = F.gelu(X @ A) @ B  # 標準答案

# Step 1: A 列切分 (各 8×6)
A1, A2 = ________________________________
# Step 2: X@Ai → GeLU (可獨立做)
H1 = ____________________________________
H2 = ____________________________________
# Step 3: B 行切分 (各 6×6)
B1, B2 = ________________________________
# Step 4: 各裝置計算部分和
Z1 = ____________________________________
Z2 = ____________________________________
# Step 5: All-Reduce (Sum)
Y_tp = __________________________________

assert torch.allclose(Y_ref, Y_tp)
print("✓ Megatron MLP TP 驗證通過！")
```

預期輸出

```
✓ Megatron MLP TP 驗證通過！
```

概念提示

第一層列切分
→ GeLU 可以獨立做

第二層行切分
→ 部分和 → All-Reduce 一次

如果第一層用行切分，
GeLU 前就必須先 All-Reduce

Part 5

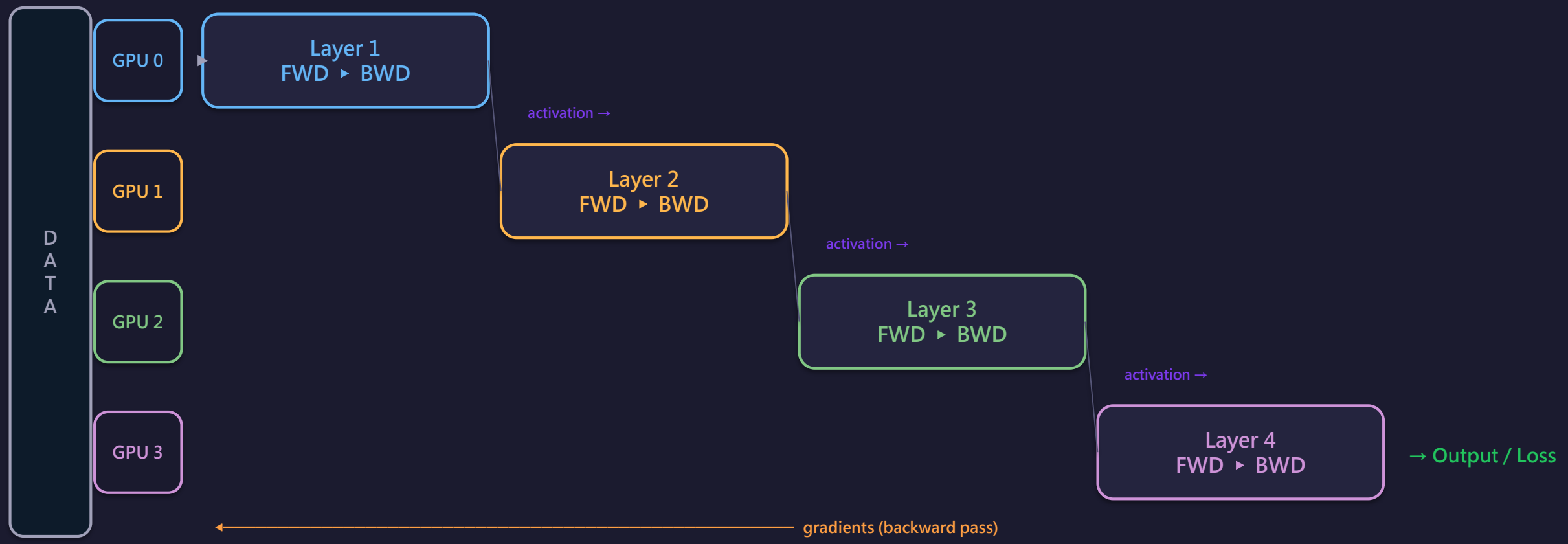
Pipeline Parallelism

管線平行 — 按層切分到不同 GPU

MP(I): Pipeline Parallelism 管線平行

按模型 Layer 切分到不同裝置，即層間平行

Model 按層切分到不同裝置 — Activation 沿裝置順序前向傳遞，Gradient 反向傳回



PP 特點：只需 P2P 通訊（前一層 → 下一層），但會產生 Pipeline Bubble（GPU 閒置）

Pipeline Parallelism — 三大排程策略

從 Naïve 到 GPipe 到 PipeDream，逐步提高 GPU 利用率

Naïve PP

1

- 最樸素的管線平行
- 同一時間只有 1 個 Device 工作
- GPU 利用率 = 1/N
- 巨大的 Pipeline Bubble

GPipe

2

- Google
- Batch → M 個 Micro-batch
- F-then-B 排程填滿管線
- Re-Materialization 省記憶體

PipeDream

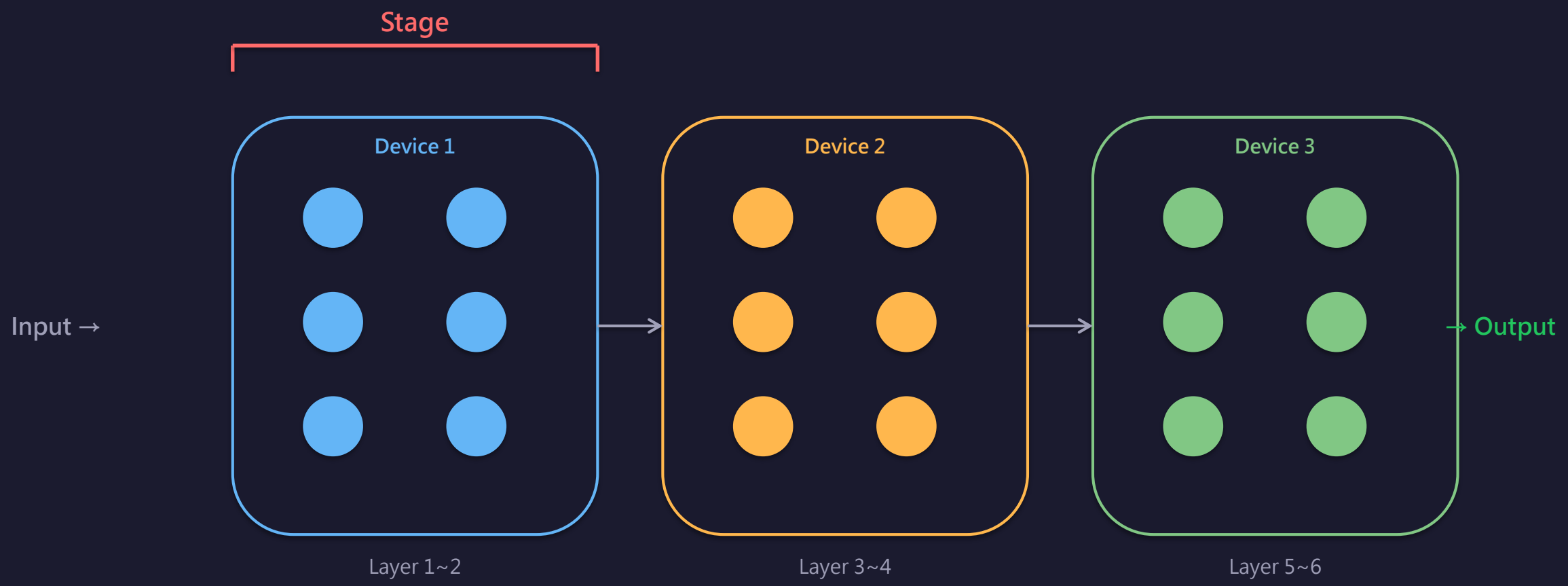
3

- Microsoft
- 1F1B：FW/BW 交替執行
- Weight Versioning 解決 staleness
- Pipeline Flush 同步 weights

PP 基本概念 — Stage

將模型按 Layer 切開，每個 Device 負責一組 Layer（= 一個 Stage）

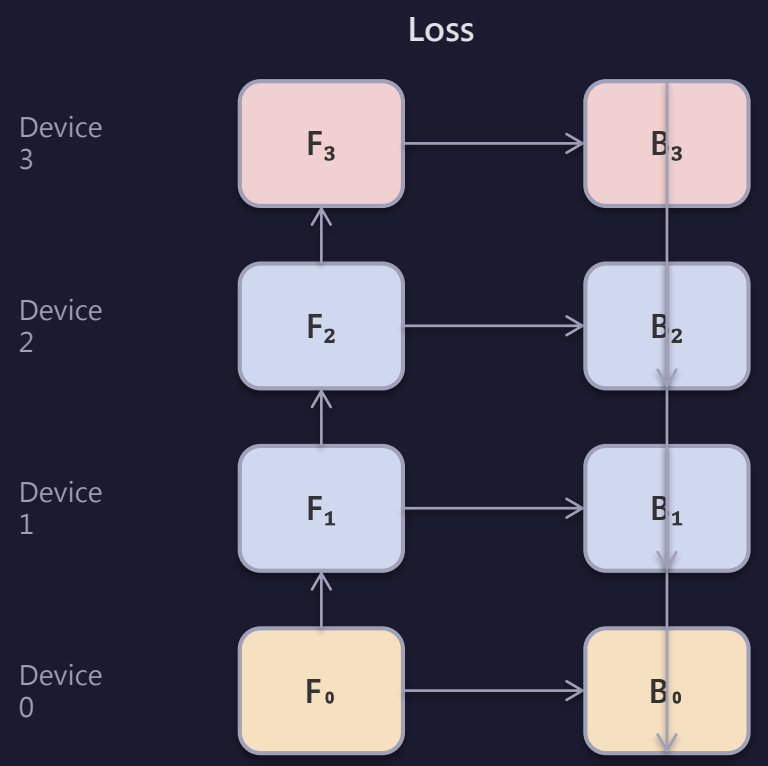
Pipeline Parallelism：按模型 layer 切分到不同 Device，即層間平行

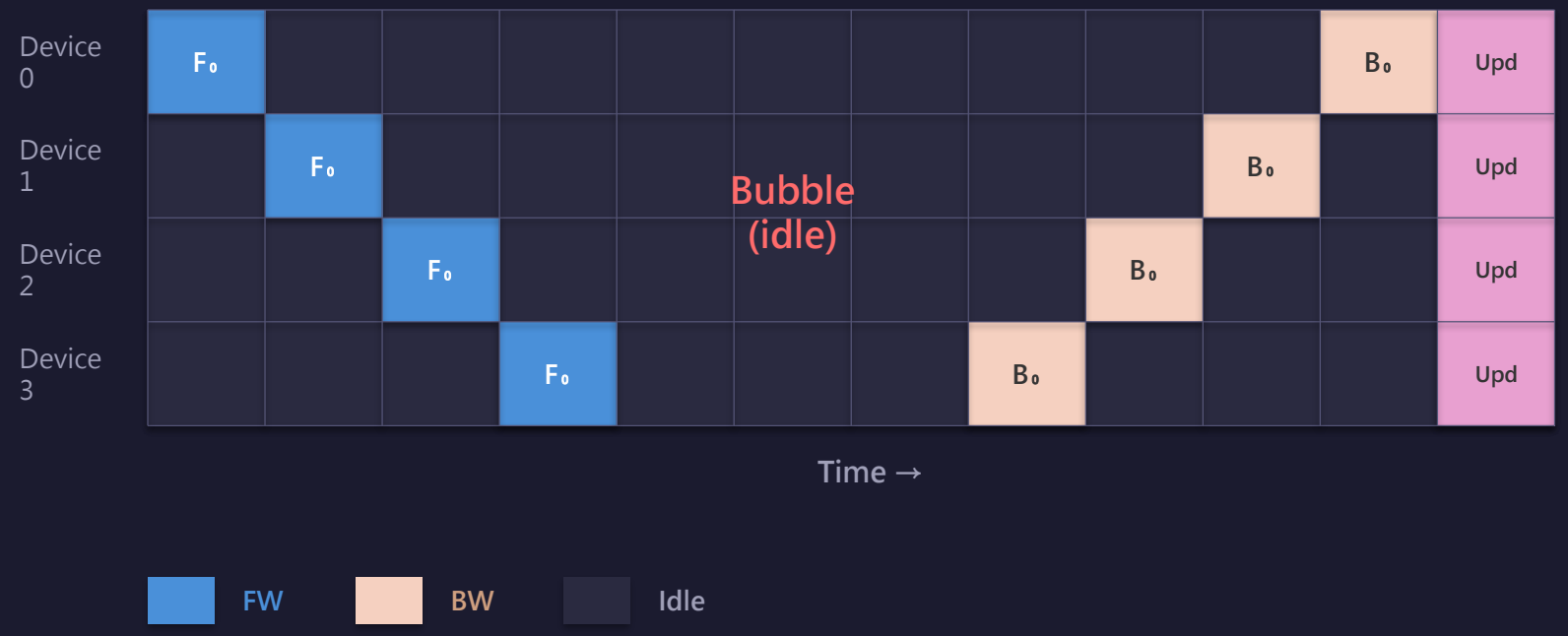


一個 Stage = 一組連續的 Layer，分配到一個 Device 上執行

Naïve Pipeline Parallelism

同一時刻只有一個 Device 在計算，其餘全部閒置



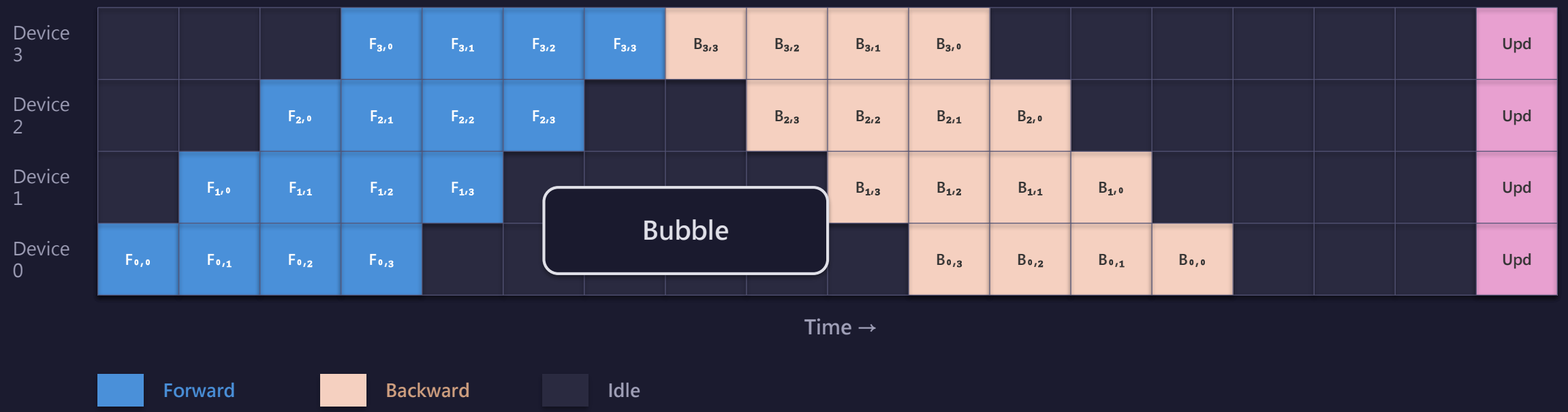


Naïve PP：GPU 利用率 = 1/N（N = Device 數量）
同一時間只有一個 Device 工作，其餘 N-1 個 Device 完全閒置 → 極度浪費

GPipe — Mini-batch Pipeline Parallelism

將 batch 切成 micro-batch，讓多個 Device 同時工作

Mini-batch PP：把 batch 再切成 M 個 micro-batch → 不同 Device 同時處理不同 micro-batch



GPipe Bubble 比例 ≈ (p−1) / m，p = Device 數，m = micro-batch 數
m 越大 Bubble 越小，但記憶體需求增加（需儲存更多中間 activation）

GPipe — 三大核心技術

Google 2019 · "GPipe: Easy Scaling with Micro-Batch Pipeline Parallelism"

1. Partition Stage

模型按 Layer 均分到 K 個 Device

- 每個 Device 負責連續的一段 Layer（一個 Stage）
- 分割策略影響負載均衡 — 各 Stage 計算量應盡量相等
- 不均衡的分割會導致更大的 Bubble

2. Micro-Batch & Pipeline

Batch 切成 M 個 Micro-batch → 填滿 Pipeline

- 大 batch 細切成 M 個小 micro-batch
- 不同 Device 同時處理不同 micro-batch → 管線化
- F-then-B 排程：先跑完所有 FW，再跑所有 BW
- 梯度在所有 micro-batch 完成後累加更新

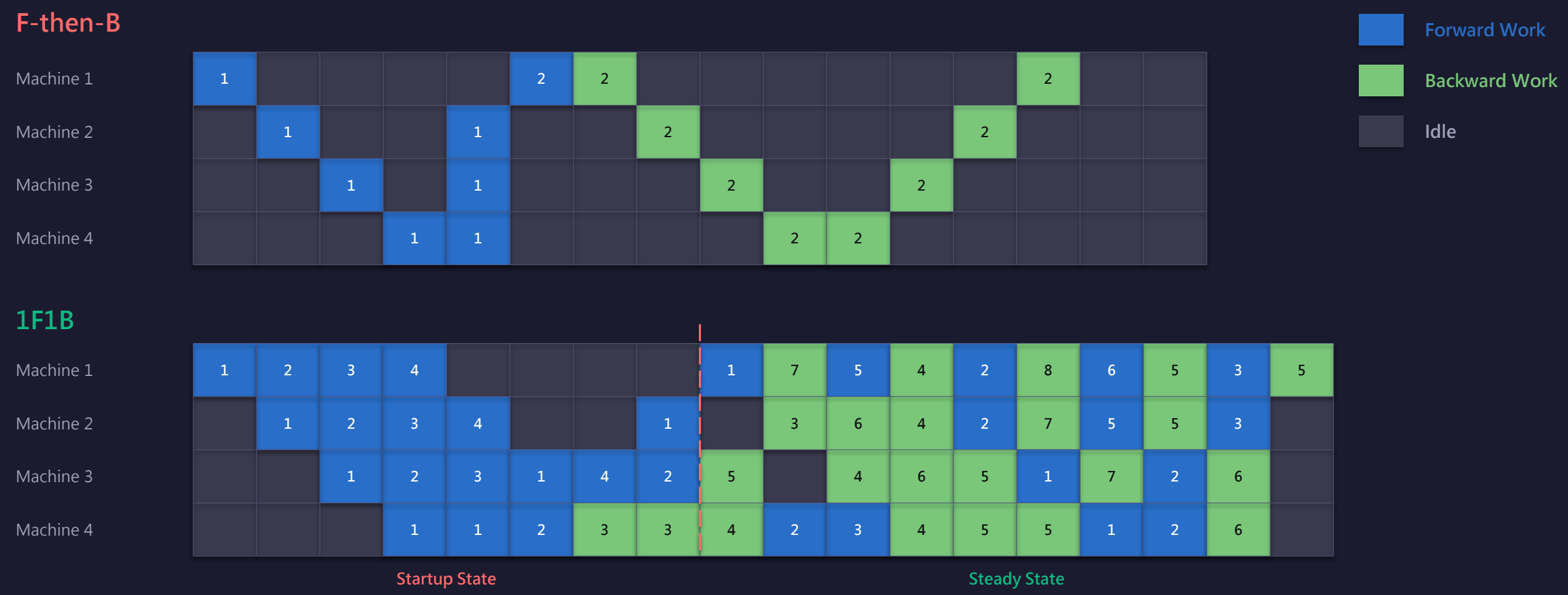
3. Re-Materialization

= Gradient Checkpointing，用時間換記憶體

- FW 時不保存中間 activation（節省記憶體）
- BW 時需要 activation → 重新跑一次 FW 來算
- 記憶體需求：O(N) → O(1)（N = Layer 數）
- 代價：多一次 FW 計算（約 33% 額外時間）

Pipeline Schedule — F-then-B vs 1F1B

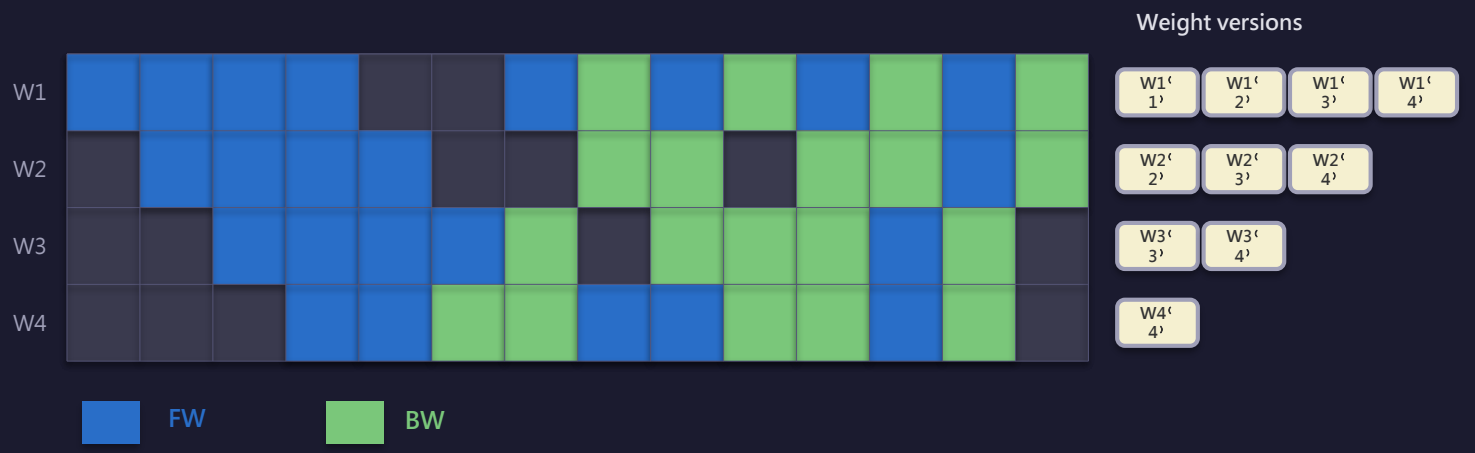
1F1B 讓 FW 和 BW 交替執行，大幅縮減 Bubble 與 Activation 記憶體



1F1B：每個 Worker 在 steady state 交替做 1 次 FW + 1 次 BW → 幾乎無 idle
相比 F-then-B，activation 記憶體從 O(m) 降為 O(p)（m=micro-batch 數, p=pipeline stage 數）

PipeDream — 1F1B + Weight Versioning

解決 1F1B 交替排程帶來的 Weight Staleness 問題



Pipeline Flush

定期在排程結尾觸發 flush：

1. 完成所有 in-flight micro-batch
2. 累加所有待處理的 gradients
3. 一次更新 weight → 所有 Worker 統一為同一版本

Flush 前：各 Worker weight 版本不同

Flush 後：全部同步為最新版本 ✓

為什麼 1F1B 需要 Weight Versioning？

問題：Weight Staleness（權重過時）

在 1F1B 交替排程中，Worker 1 做 FW(micro-batch 5) 時，BW(1)、BW(2)、BW(3)、BW(4) 可能已經更新過 weights 了。

→ FW(5) 用的 weight 跟 BW(5) 用的 weight 可能不同版本！

PipeDream 的解法：Weight Versioning

每個 Worker 保留多份 weight buffer（$W^{(1)}$, $W^{(2)}$, $W^{(3)}$ ...）

做 FW(k) 時記錄當前 weight 版本 → BW(k) 時取出同版本計算

✓ 確保同一 micro-batch 的 FW 和 BW 使用一致的 weight

練習作業 4.1

Naïve Pipeline Parallelism 模擬

情境說明

PP 將模型按層切分到不同裝置。本題模擬 4 層 MLP 分配到 4 個 Stage，驗證逐 Stage 前向傳遞的結果與單卡一致。

程式骨架

```
import torch
import torch.nn as nn
torch.manual_seed(42)
X = torch.randn(4, 8)
target = torch.randn(4, 2)
layers = [nn.Linear(8,6), nn.Linear(6,6),
          nn.Linear(6,4), nn.Linear(4,2)]
# 單卡標準答案
Y_ref = X
for layer in layers:
    Y_ref = torch.relu(layer(Y_ref))

# 任務 1: 每層分配給一個 Stage
stage_0 = ______________________________
stage_1 = ______________________________
stage_2 = ______________________________
stage_3 = ______________________________
# 任務 2: 逐 Stage 前向傳遞
act_0 = ________________________________
act_1 = ________________________________
act_2 = ________________________________
Y_pp  = ________________________________

assert torch.allclose(Y_ref, Y_pp)
print("✓ Pipeline 前向傳遞驗證通過！")
```

預期輸出

```
✓ Pipeline 前向傳遞驗證通過！
Stage 0 參數量: 48
Stage 3 參數量: 8
```

概念提示

每個 Stage 只「看得到」
自己那一層

前向傳遞的 activation
就像接力棒一樣傳遞

想想看：4 Stage 情況下
同時有幾個 GPU 在工作？

練習作業 4.2

GPipe Micro-batch Pipeline 模擬

情境說明

GPipe 把 batch 切成多個 micro-batch，讓不同 Stage 交錯計算。本題模擬 micro-batch 切分與 loss 聚合。

程式骨架

```
import torch
import torch.nn as nn
torch.manual_seed(42)
X = torch.randn(8, 6)
target = torch.randn(8, 2)
stage_1 = nn.Linear(6, 4)
stage_2 = nn.Linear(4, 2)
# 單卡標準答案
Y_ref = stage_2(torch.relu(stage_1(X)))
loss_ref = nn.MSELoss()(Y_ref, target)

M = 4  # micro-batch 數量
# 任務 1: 切成 M 個 micro-batch
X_micros = ______________________________
T_micros = ______________________________
# 任務 2: 逐 micro-batch 過 pipeline
total_loss = torch.tensor(0.0)
for i in range(M):
    h = torch.relu(stage_1(X_micros[i]))
    out = stage_2(h)
    micro_loss = ________________________
    total_loss = total_loss + micro_loss
print(f"GPipe loss: {total_loss.item():.4f}")
```

預期輸出

```
Single loss: X.XXXX
GPipe  loss: X.XXXX
```

概念提示

把大 batch 切成 M 小份
分別走 pipeline

多份 micro-batch 的 loss
合併時要注意量級問題

MSELoss 對每份做 mean
跨份要怎麼處理？

Part 7

混合式訓練

3D/4D Parallelism — 組合使用多種平行策略

為什麼需要混合式訓練？

Why Hybrid Parallelism? — 單一策略的天花板

只用 DP / FSDP

模型必須放得進一張 GPU
All-Gather 完整參數 → 175B+ 放不下
All-Reduce 通訊量 ∝ 模型大小
極限：約幾十 B 參數

只用 TP

每層 2 次 All-Reduce → 通訊極頻繁
96 層 = 192 次 All-Reduce / step
必須用 NVLink → 限制在同節點
極限：TP ≤ 8（一台機器）

只用 PP

Pipeline Bubble 浪費 10-25% 算力
不切層內參數 → 單層太大無解
Micro-batch 管理複雜
極限：利用率 ~80%

解法：組合使用，各取所長

TP 切開單層
解決一層太大

PP 切開不同層
解決層數太多

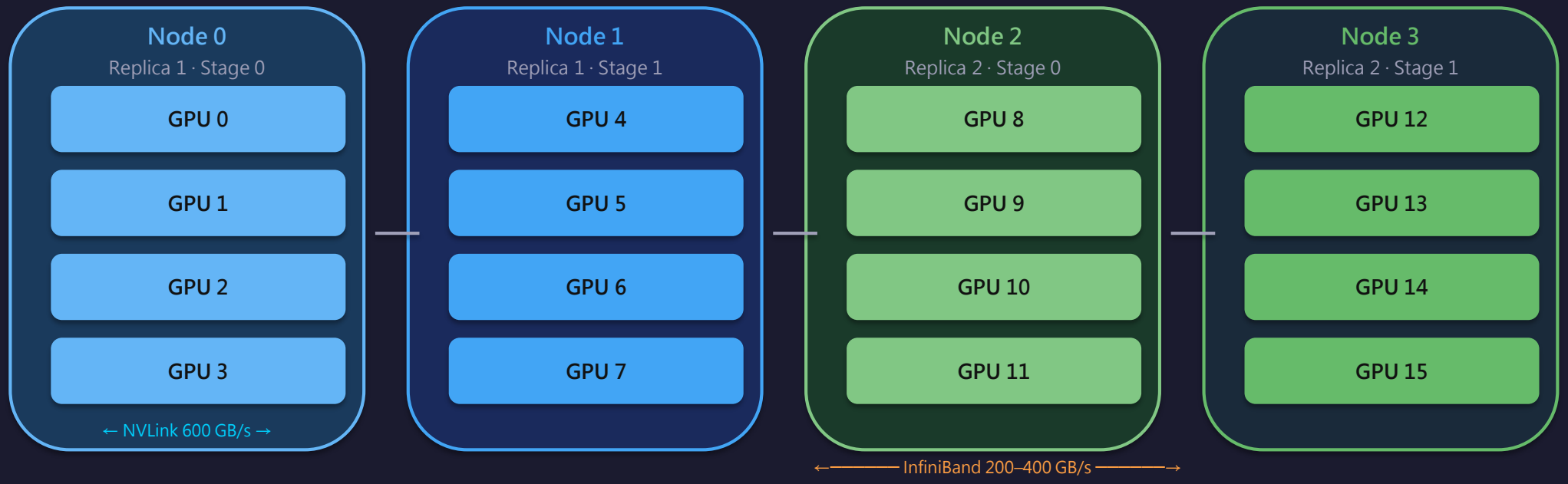
DP 切開資料
解決速度不夠

Total GPUs
= DP × TP × PP

3D Parallelism = DP × TP × PP

把三種策略分配到 GPU 集群的不同連結層級

範例：16 GPUs（4 台機器 × 4 GPUs），設定 TP=4, PP=2, DP=2



TP Group

同節點 4 GPUs → All-Reduce

PP Group

Node 0→1 或 2→3 → P2P

DP Group

Node 0↔2, 1↔3 → All-Reduce

3D Parallelism 通訊分析

不同維度的通訊操作、通訊量、頻率與硬體需求

維度	通訊操作	每次通訊量	頻率	最佳硬體
TP (張量平行)	All-Reduce	2 × hidden² × bs × seq / TP	每層 2 次 (MLP + Attn) 80 層 → 320 次/step	NVLink (600 GB/s)
PP (管線平行)	P2P Send/Recv	bs × seq × hidden (activation 大小)	每 micro-batch 在 stage 邊界傳 1 次	InfiniBand
DP (資料平行)	All-Reduce (或 RS + AG)	2 × 參數量 / TP / PP	每 grad accum 之後 1 次 可與計算重疊	InfiniBand

硬體拓撲匹配

- NVLink (最快) → 給 TP
- InfiniBand → 給 PP + DP
- 口訣：TP 吃頻寬，PP 吃延遲，DP 吃次數

Sequence Parallelism & Context Parallelism

沿序列維度切分 — 從 3D 到 4D Parallelism

Sequence Parallelism (SP)

Megatron 提出，解決 TP 的 activation 重複問題

問題：TP 只切了 MLP / Attention 的權重
但 LayerNorm / Dropout 的 activation 每張卡都存了完整一份

SP 做法：
非 TP 層的 activation 沿 sequence 維度切分
與 TP 共用同一 Process Group，不加 GPU
All-Reduce → 拆成 RS + AG，中間插入計算
→ 全部 activation 零重複

Context Parallelism (CP)

專為超長序列設計（128K+ tokens）

問題：Attention score = seq × seq
128K 序列 → 16B 個數字 → 單卡放不下

CP 做法：Ring Attention
所有 activation 沿序列維度切分
GPU 排成環形，KV cache 沿環傳遞
每張 GPU 用自己的 Q 和收到的 KV 計算
→ 序列長度可擴展到 1M+

4D Parallelism

Total GPUs = DP × TP × PP × CP

序列不長（< 8K）→ 不需要 CP
128K+ 上下文 → CP 是必要的
SP 省記憶體但不加 GPU，CP 需要額外 GPU

實戰案例：Llama-3.1 & DeepSeek-V3

真實大模型的混合式訓練配置

Llama-3.1 405B (Dense)

模型	405B 參數，126 層，hidden=16384
GPU	16,384 × H100 (2048 nodes)
TP	= 8（一台機器全給 TP，NVLink）
PP	= 16（126 層 ÷ ~8 層/stage）
DP	= 128（16384 ÷ 8 ÷ 16）
CP	有（支援 128K 上下文）
訓練	~54 天，~30.8M GPU-hours

TP=8 正好 = 一台機器卡數
PP=16 把 126 層切成 16 段

DeepSeek-V3 671B (MoE)

參數	671B 總量，37B 活躍 / token
架構	MoE，256 個 Expert
GPU	2,048 × H800
TP + EP	TP + Expert Parallelism
EP 做法	Expert 分散在不同 GPU
通訊	All-to-All 做 token 路由
成本	$5.6M（GPT-4: ~$100M）

MoE：參數多但每次只用 5%
EP 用 All-to-All 路由 token 到 Expert

混合式訓練調參指南

如何決定 TP / PP / DP 的大小？— 五步決策流程

Step 1 模型放得進一張 GPU 嗎？

參數量 × 16 bytes（FP16 + Adam）
放得進 → DDP / FSDP 即可

Step 2 放不進 → 先設 TP

TP ≤ 節點內 GPU 數（通常 8）
模型越大 TP 越大：70B→TP=4-8

Step 3 還放不下 → 加 PP

PP = Total_layers / layers_per_GPU
PP 越小越好（減少 bubble）

Step 4 DP = Total / (TP × PP)

自動算出，DP 越大 throughput 越高
注意 global batch size 調整

Step 5 序列超長 → 加 CP

seq_len > 32K 且 activation 爆了
CP=2 把序列切半

常見作法

- TP ≤ 節點內 GPU 數
- PP 越小越好（2-4 常見）
- DP 越大越好（線性加速）
- 先試 FSDP + TP
- 不夠再加 PP

練習作業 5.1

混合式訓練策略選擇 — 開放討論

情境 A

你有 1 台機器、8 張 GPU，GPU 之間可透過 NVLink 互連。
現在要訓練一個 7B 的 dense 模型。
已知這個模型大致可以放進單張 GPU，訓練資料量中等，希望提升整體訓練效率。

請回答：

你會選擇哪一種平行策略？

可考慮：DP / DDP / FSDP / TP / PP，也可以組合使用

請畫出你的 GPU 分組示意圖

請簡單說明你的理由

至少回答：

- 為什麼這個情境可能不一定需要 TP 或 PP
- 你選擇 DDP 或 FSDP 的原因是什麼

提示

這題沒有正解

只要你的設計合理、說明清楚即可

思考方向：

- 7B 模型放得進一張 GPU → 需要 TP 嗎？
- 8 張卡都在同一台機器 → 有跨節點通訊嗎？
- DDP vs FSDP：各自的優缺點是什麼？
- 7B × 16 bytes ≈ 112GB → 超出 80GB

總結

Summary

集合通訊

Broadcast / Scatter / Reduce / Gather
All-Reduce = Reduce-Scatter + All-Gather
All-to-All：MoE 路由核心
所有分散式策略的通訊基礎

張量平行 TP

切分層內權重（Megatron 風格）
列平行 + 行平行 + All-Reduce
MLP / Self-Attention 均可切分
適合節點內 NVLink 高速互聯

資料平行 DP / DDP / FSDP

DP：主節點聚合梯度（瓶頸）
DDP：Ring All-Reduce + 通算重疊
FSDP/ZeRO：參數切片 + 按需 All-Gather

管線平行 PP

按層切分到不同 GPU
Naïve → GPipe → 1F1B → Interleaved
只需 P2P 通訊，適合跨節點

混合式訓練

3D Parallel = DP × TP × PP
TP→NVLink ｜ PP→InfiniBand ｜ DP→剩餘
4D = + Sequence / Context Parallelism

Thank You!

分散式訓練